Department of Computer Science and Engineering

MODULE 3

Data Link Layer

Computer Networks (PCCST501)

APJ Abdul Kalam Technological University (KTU) — 2024 Scheme

Ms. ANJU MARKOSE,
Asst. Professor & Dept. Secretary,
Dept. of Computer Science and Design,
VJCET, Vazhakulam.
Ph No: 9946004607

1. Introduction to the Data Link Layer

1.1 Position in the TCP/IP Protocol Stack

The Data Link Layer (also called the Link Layer or Network Access Layer) is the lowest layer of the TCP/IP protocol stack that deals with logical communication — the Physical Layer below it only deals with raw bits and signalling. The Data Link Layer is responsible for transmitting data between two directly connected (adjacent) devices over a single physical link, such as a cable, an optical fibre, or a radio channel.

The Data Link Layer receives packets from the Network (Internet) Layer above it and encapsulates them into frames before handing them to the Physical Layer for transmission. At the receiving end, the Data Link Layer strips off the frame header and trailer that were added by the sender, checks the frame for errors, and passes the enclosed packet up to the Network Layer. This encapsulation/decapsulation is what allows the layer above (Network Layer, dealing with IP addressing and routing) to remain unaware of the physical medium being used underneath it.

Functions of the Data Link Layer

- Provides hop-to-hop (node-to-node) communication — as opposed to the Transport Layer, which provides end-to-end (host-to-host) communication.
- Performs framing — converts the stream of packets/bits into recognisable frames with clear start and end boundaries.
- Uses MAC (physical) addresses to identify devices on the same link.
- Detects (and sometimes corrects) transmission errors, typically using CRC.
- Controls access to the transmission medium when the medium is shared among multiple devices (Media Access Control).

1.2 Nodes and Links

In the Data Link Layer, communication is described as node-to-node or hop-to-hop, since data does not go directly from the original sender to the final receiver in one shot; instead, it travels through a chain of intermediate nodes connected by links, being re-framed at every hop.

- Nodes: the source host, the destination host, and every router in between.
- Links: the physical networks (LANs or WANs) that connect two adjacent nodes.

Data is transmitted one link at a time. At each node, the incoming frame is decapsulated to retrieve the packet, and a brand-new frame (with new Data-Link-Layer addressing, appropriate for the next link) is

created before the packet is forwarded onward — the packet itself is untouched, but the frame around it changes at every hop.

Example: Source Host → Router → Router → Router → Destination Host. Here, the hosts and routers are the nodes, and the individual networks connecting consecutive nodes are the links.

1.3 Two Types of Links

Depending on how many devices share the physical channel, a link used by the Data Link Layer can be classified into two broad types:

1. Point-to-Point Link

- A dedicated link exists between exactly two devices.
- The entire capacity of the channel belongs only to those two devices — there is no sharing and, therefore, no possibility of a collision with a third party.
- Example: two routers connected by a leased line, or a computer connected to its ISP via a telephone/DSL line.

2. Broadcast (Multipoint) Link

- A single shared link is used by more than two devices at the same time.
- Because the medium is common, a mechanism is needed to decide which device may transmit at a given instant — this is the job of the Media Access Control (MAC) sublayer discussed later in this module.
- Example: a wired Ethernet LAN using a shared bus/hub, or a Wi-Fi network where every station shares the same radio channel (the air).

1.4 Two Sublayers of the Data Link Layer

Because a Data Link Layer protocol must potentially deal with both kinds of links described above, and because broadcast links additionally require medium-access coordination, the Data Link Layer is conceptually divided into two sublayers:

1. Data Link Control (DLC)

DLC handles functions that are common to both point-to-point and broadcast links — essentially everything needed to move a frame reliably across one link, regardless of whether that link is shared or not. Its responsibilities are framing, flow control, and error control (detection and, in some cases, correction).

2. Media Access Control (MAC)

MAC handles functions that are specific to broadcast (shared) links only. It decides, using a multiple-access protocol, which of the several competing stations gets to use the shared medium at a given time, and it is also where MAC (physical) addressing is used to identify individual stations on the shared link.

> *Note: Point-to-point links need only the DLC sublayer (no contention exists), while broadcast links need both DLC and MAC sublayers working together.*

2. Data Link Control (DLC): Framing

2.1 Why Framing Is Needed

The Physical Layer only understands and transmits a raw stream of bits — it has no notion of where one 'message' ends and the next begins. The Data Link Layer imposes structure on this stream by grouping bits into frames, each with a clearly marked header and trailer. The header typically carries control information such as the source and destination MAC addresses, while the trailer typically carries error-detection bits (such as a CRC). Framing lets the receiver identify exactly where each frame starts and ends, which is essential for reliable communication.

Rather than sending an entire message as one giant, continuous transmission, the message is broken into a sequence of smaller frames. This granularity directly improves both flow control and error control: if a single frame is lost or corrupted, only that one frame needs to be retransmitted, instead of the entire (possibly very large) message having to be sent again.

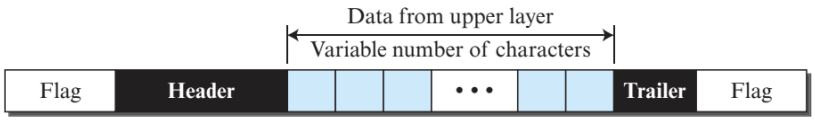
2.2 Frame Size: Fixed vs. Variable

- Fixed-size framing: every frame has exactly the same length, so the receiver automatically knows where one frame ends and the next begins purely by counting bits/bytes. Example: ATM (Asynchronous Transfer Mode), which uses fixed-size frames called cells (53 bytes each).
- Variable-size framing: frames may have different lengths, so an explicit technique is required to mark frame boundaries. Two such techniques are in common use — character-oriented (byte-oriented) framing and bit-oriented framing.

2.3 Character-Oriented (Byte-Oriented) Framing

In character-oriented framing, both the data and any control information are represented using 8-bit characters (bytes), typically from a coding system such as ASCII. A special reserved byte, called the flag, is placed at the very beginning and at the very end of every frame, so the receiver can immediately recognise where a frame starts and stops.

Figure 5.4 *A frame in a character-oriented protocol*



A frame in a character-oriented protocol: Flag – Header – variable data – Trailer – Flag.

A problem arises when the flag byte's exact bit pattern happens to occur naturally inside the data being carried — the receiver could then mistakenly interpret it as the end of the frame and truncate the message. This is solved using Byte Stuffing (also called Character Stuffing): the sender inserts a special Escape (ESC) byte immediately before any byte in the data that coincidentally matches the flag pattern. When the receiver sees the ESC byte, it discards the ESC and treats the very next byte as ordinary data rather than as a flag.

A similar issue can occur if the ESC character itself appears naturally within the data — this is handled the same way, by stuffing an additional ESC byte before it. The receiver, upon seeing two consecutive ESC bytes, removes only the first one and correctly interprets the second as literal data.

Although character-oriented framing was historically popular (particularly for text-based communication), it is poorly suited to modern multimedia data such as graphics, audio and video, and to larger character sets such as Unicode, where reserving specific byte patterns as flags/escapes becomes awkward. For this reason, it has largely been replaced by bit-oriented framing in modern protocols.

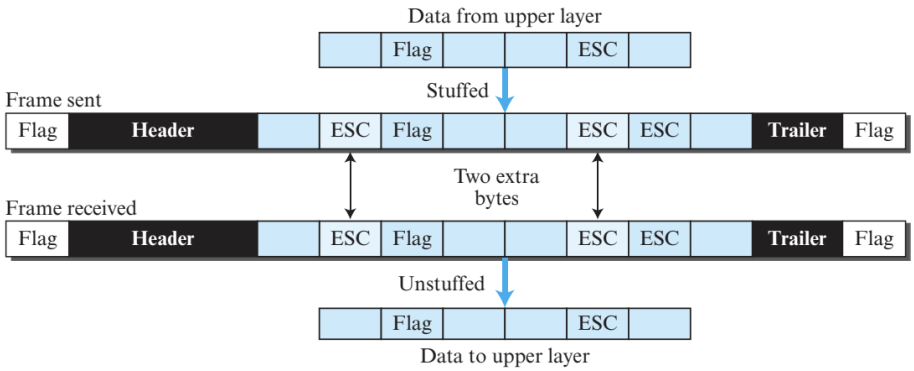
2.4 Bit-Oriented Framing

In bit-oriented framing, a frame is viewed simply as a sequence of bits carrying data that may have some semantic meaning to the upper layers (text, audio, video, and so on), rather than as a sequence of 8-bit characters. Most bit-oriented protocols (e.g., HDLC) use a special 8-bit flag pattern, 01111110, to mark both the beginning and the end of a frame.

If this exact bit pattern happens to occur inside the data itself, it could again be mistaken for a frame boundary. This is prevented using Bit Stuffing: whenever the sender's hardware detects five consecutive 1 bits in the outgoing data stream, it automatically inserts an extra 0 bit immediately after them. Since the flag pattern 01111110 contains six consecutive 1s only never five-then-continue, this insertion guarantees that the six-bit pattern of the flag can never occur naturally inside the stuffed data.

At the receiving end, the reverse operation, bit destuffing, is performed: whenever the receiver detects five consecutive 1 bits followed by a 0, it strips out that inserted 0 bit, restoring the original data exactly. As a result, the flag sequence 01111110 can only appear at the true beginning and end of a frame, allowing the receiver to reliably distinguish genuine frame boundaries from ordinary data.

Figure 5.5 *Byte stuffing and unstuffing*



Byte stuffing is the process of adding one extra byte whenever there is a flag or escape character in the text.

Bit stuffing and destuffing: an extra 0 is inserted after every run of five consecutive 1s, and removed at the receiver.

3. Flow Control and Error Control

3.1 Flow Control

Flow control is the process of regulating the rate at which the sender transmits data so that a fast sender does not overwhelm a slower receiver. Without flow control, if the sender's transmission rate exceeds the rate at which the receiver's buffer can process incoming frames, the receiver's buffer will overflow and the excess frames will simply be lost.

At the Data Link Layer, flow control is node-to-node (hop-to-hop): it coordinates, for a single link, how many frames the sender is allowed to transmit before it must pause and wait for an acknowledgment from the immediate next node. This is different from flow control at the Transport Layer, which is end-to-end (host-to-host) and coordinates data flow across the entire path from the original source to the final destination.

3.2 Error Control

Error control ensures the reliable delivery of frames across a link by detecting transmission errors and, where required, recovering from them (typically through retransmission). It therefore covers both error detection and error correction/retransmission.

At the Data Link Layer, whenever a frame is found to be lost or corrupted on a given link, the receiving node informs the sending node (on that same link), and the sender retransmits only the affected frame rather than the whole message. Like flow control, error control at this layer is node-to-node: every time a frame crosses one link, it is checked for errors before being forwarded further towards its destination. This differs again from the Transport Layer, where error control (for example, in TCP) is performed end-to-end.

4. Error Detection and Correction

Error Detection and Correction is a Data Link Control (DLC) function that ensures reliable transmission of frames between two adjacent nodes. If a frame is corrupted while crossing a link, the corruption must be detected — and, ideally, corrected — before the frame is forwarded further. In practice, most wired link-layer protocols only detect errors and simply discard corrupted frames, leaving retransmission to a higher-layer protocol (such as TCP); some wireless protocols, where retransmission over the air is expensive, go a step further and attempt to correct errors directly.

4.1 Types of Errors

During transmission, electrical noise, interference, or attenuation can corrupt one or more bits of a frame. Errors are broadly classified into two types:

1. Single-Bit Error

A single-bit error occurs when exactly one bit in the transmitted data changes value — a 0 becomes a 1, or a 1 becomes a 0 — while every other bit arrives correctly.

2. Burst Error

A burst error occurs when two or more consecutive bits in the data are corrupted together. Burst errors are far more common in practice than single-bit errors, because real-world noise (such as a lightning strike, a switching transient, or a fade in a wireless channel) usually affects a contiguous stretch of the signal, corrupting several consecutive bits rather than just one isolated bit. The length of a burst error is measured

from the first corrupted bit to the last corrupted bit in the run, inclusive of any bits in between that may or may not also be in error.

4.2 Redundancy

Redundancy is the fundamental idea underlying essentially every error detection and correction scheme: the sender adds extra bits, called redundant bits, to the original data before it is transmitted. These redundant bits do not carry any new information by themselves; instead, they encode a known mathematical relationship with the original data bits. The receiver re-checks this relationship on the bits it actually received, and any violation of the expected relationship signals that an error has occurred. Once the check is complete, the redundant bits are discarded and only the original data is passed up to the next layer.

4.3 Error Detection vs. Error Correction

Aspect	Error Detection	Error Correction
What it tells us	Only whether an error occurred (Yes/No)	Which bit(s) are wrong, and how to fix them
Complexity	Comparatively simple	More complex — needs to pinpoint both the number and the exact position of the erroneous bits
Typical action on error	Discard the frame; rely on retransmission	Flip the identified bit(s) back to their correct values

Because error correction must additionally determine the precise location of every corrupted bit before it can be fixed, it is inherently more complex — and typically requires far more redundant bits — than simple error detection.

4.4 Coding

Coding is the general process of adding redundant bits to data according to a specific mathematical rule (the 'code'), so that the receiver can later verify whether the rule still holds for the bits it received. Coding techniques are broadly classified into Block Coding and Convolutional Coding; of these, Block Coding is by far the more common technique at the Data Link Layer, and is the one covered in this module.

Block Coding

In block coding, the message to be sent is first divided into fixed-size blocks of k data bits each, called datawords. Each k-bit dataword is then transformed into a longer, n-bit codeword by appending r redundant bits, where:

$$n = k + r$$

- Dataword: the original k bits of actual data.
- Codeword: the dataword together with its r redundant bits, giving n bits in total.

Out of the 2^n possible n-bit patterns, only 2^k of them are designated as valid codewords (these are the ones the encoder is actually allowed to produce); the remaining patterns are, by design, invalid. When the receiver gets a codeword, it simply checks whether that pattern belongs to the set of valid codewords:

- Valid codeword → the data is accepted (assumed error-free).

- Invalid (illegal) codeword → an error is deemed to have occurred during transmission, and the frame is rejected.

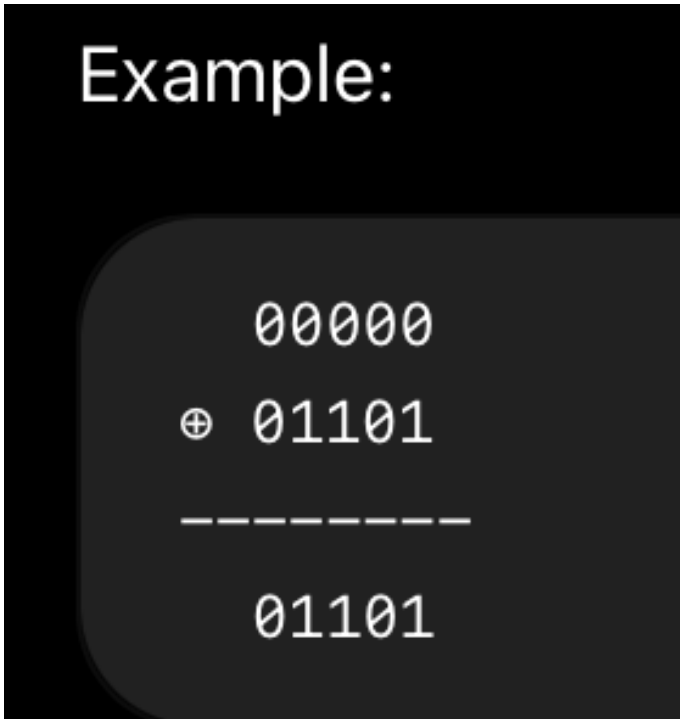
Note: If noise happens to corrupt a valid codeword into a pattern that is, by coincidence, also a valid codeword, the error goes completely undetected. Good codes are therefore designed so that valid codewords are 'far apart' from one another (see Hamming distance below), making such coincidental corruption very unlikely.

5. Hamming Distance

The Hamming Distance between two binary words of the same length is defined as the number of bit positions at which the two words differ. It is written as d(x, y), and is the key mathematical tool used to reason about how many errors a given code can reliably detect or correct.

Example: let the sent codeword be 00000 and the received codeword be 01101.

Comparing bit by bit: position 2 differs (0 vs 1), position 3 differs (0 vs 1), and position 5 differs (0 vs 1) — three positions in total differ. Therefore d(00000, 01101) = 3, meaning exactly 3 bits were corrupted in transit.



Hamming distance is conveniently computed by XOR-ing the two words and counting the number of 1s in the result.

A convenient computational shortcut: the Hamming distance between two words equals the number of 1s in the bitwise XOR (⊕) of the two words. XOR-ing 00000 with 01101 gives 01101, which has three 1-bits, confirming d = 3 without any need to compare positions manually.

5.1 Minimum Hamming Distance for Error Detection

The minimum Hamming distance (d_{min}) of a code is the smallest Hamming distance found between any two valid codewords in the entire codeword set. It represents the 'closest' that any two legitimate codewords ever get to each other, and therefore determines the code's worst-case error-detecting ability.

$$\textbf{To detect up to s errors reliably: } d_{min} = s + 1$$

This relationship guarantees that a codeword corrupted by s or fewer bit errors can never accidentally land on top of another valid codeword — it will always fall into 'invalid' territory and therefore be caught.

Worked example: suppose the design goal is to reliably detect up to 2 errors in every codeword. Then s = 2, so the code must be designed with $d_{min} = 2 + 1 = 3$. In other words, every pair of valid codewords in this code must differ in at least 3 bit positions.

> *Note: The rule '$d_{min} = s + 1$' guarantees detection of s or fewer errors. It is possible, but not guaranteed, that some error patterns with more than s errors are also detected — the code simply makes no promises beyond s errors.*

6. Linear Block Codes

A Linear Block Code is a special, widely used category of block code with one extra structural property: the XOR (modulo-2 addition) of any two valid codewords in the code is itself always another valid codeword. This 'closure' property makes linear block codes considerably easier to analyse, design, and implement in hardware compared with arbitrary (nonlinear) block codes, which is why they dominate practical use.

A useful consequence of linearity is that the minimum distance of a linear block code equals the minimum number of 1s found in any single nonzero valid codeword — there is no need to compare every pair of codewords against each other. For example, if the nonzero valid codewords of a code contain 2, 2, and 3 ones respectively, then $d_{min} = 2$ (the smallest count observed).

6.1 Simple Parity-Check Code

The Parity-Check Code is the simplest and most commonly taught linear block code. A k-bit dataword is converted into an n-bit codeword by appending exactly one extra bit, called the parity bit, so that:

$$n = k + 1$$

For even parity (the usual convention), the parity bit is chosen so that the total number of 1s across the entire codeword (data bits plus parity bit) is always even. The minimum Hamming distance of a simple parity-check code is $d_{min} = 2$ — which, by the $s = d_{min} - 1$ relationship, means it can reliably detect any single-bit error (and, more generally, any odd number of bit errors).

For a 4-bit dataword (a_3, a_2, a_1, a_0), one parity bit r_0 is added to produce a 5-bit codeword, where the parity bit is computed by XOR-ing all four data bits:

$$r_0 = a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

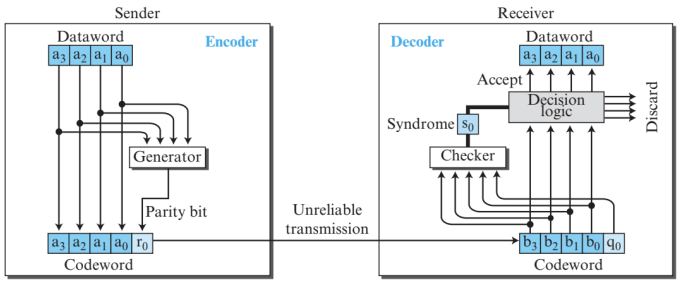
If the dataword already contains an even number of 1s, $r_0 = 0$; if it contains an odd number of 1s, $r_0 = 1$. Either way, the resulting 5-bit codeword always contains an even total count of 1s.

Table 5.2 *Simple parity-check code C(5, 4)*

Datawords	Codewords	*Datawords*	Codewords
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Figure 5.11 shows a possible structure of an encoder (at the sender) and a decoder (at the receiver).

Figure 5.11 *Encoder and decoder for simple parity-check code*



Encoder and decoder for a simple parity-check code — the encoder computes the parity bit r0; the decoder recomputes the syndrome s0.

Worked Example: Encoding

Dataword = 1011. Number of 1s in the dataword = 3 (an odd number). Therefore the parity bit must be 1, to make the total even. Codeword = 10111. Checking: the codeword 10111 contains four 1-bits, which is indeed even, so the even-parity condition is correctly satisfied.

Encoder and Decoder

At the sender, an encoder (generator circuit) takes the k-bit dataword and computes the parity bit as described above; the dataword and the parity bit are combined to form the codeword, which is then transmitted across the channel. At the receiver, a checker (decoder circuit) receives the codeword and XORs together all of the received bits, including the received parity bit. This result is called the syndrome.

$$s_0 = b_3 \oplus b_2 \oplus b_1 \oplus b_0 \oplus q_0$$

- Syndrome = 0: the total number of 1s received is even, so no error is detectable; the data portion is accepted as the dataword.
- Syndrome = 1: the total number of 1s received is odd, so an error has definitely been detected; the received data is discarded.

Worked Example: Error Detection

Suppose the sender transmits the codeword 10111 (from the example above).

- No error: receiver gets 10111 → syndrome = 0 → accept → recovered dataword = 1011.
- One bit flips in transit: receiver gets 10011 → syndrome = 1 → error detected → frame discarded.

Limitation of Simple Parity Check

A simple parity-check code is guaranteed to detect every single-bit error, and more generally every case where an odd number of bits are corrupted, since flipping an odd number of bits always changes the overall parity. However, it cannot reliably detect an even number of simultaneous bit errors, because two (or any even number of) flipped bits can cancel each other out in the parity calculation, producing a syndrome of 0 even though the data is actually corrupted. For example, if exactly two bits happen to flip, the received codeword may still contain an even total number of 1s; the receiver computes syndrome = 0 and wrongly accepts the corrupted data as correct. This is precisely why $d_{min} = 2$ only guarantees detection of $s = 1$ error, i.e., single-bit errors.

7. Cyclic Codes and CRC

7.1 Cyclic Codes

Cyclic Codes are a special subclass of linear block codes with a further structural property: if any valid codeword is cyclically shifted (rotated) by any number of positions, the result is also a valid codeword of the same code.

Example: Codeword = 1011000. After a cyclic left shift by one position: 0110001. Since this rotated pattern is also a member of the valid codeword set, the code is called cyclic.

7.2 Cyclic Redundancy Check (CRC)

The Cyclic Redundancy Check (CRC) is by far the most widely used error-detection technique based on cyclic codes. It is used extensively in LANs, WANs, Ethernet, USB, and many other communication protocols to detect transmission errors reliably and with low computational overhead.

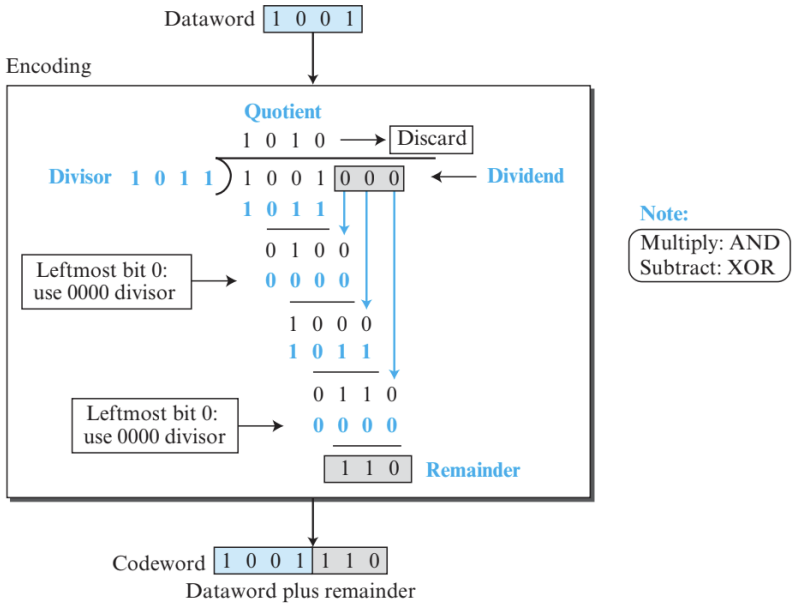
- A k-bit dataword is converted into an n-bit codeword.
- (n − k) redundant bits, called the CRC bits, are appended to the dataword.
- CRC is capable of detecting single-bit errors, double-bit errors, burst errors, and many other multiple-bit error patterns, with very high (near-certain) accuracy.

CRC Encoder (Generating the CRC bits)

At the sender, the encoder derives the CRC bits through the following steps:

- Take the k-bit dataword.
- Append (n − k) zero bits to the end of the dataword — this creates an 'augmented' dataword of n bits.
- Divide this augmented dataword by a predefined generator (divisor) polynomial, using Modulo-2 (XOR) division.
- Discard the quotient of this division entirely — it is not used.
- The remainder obtained from the division becomes the CRC bits.
- Replace the appended zeros with this remainder to form the final n-bit codeword, which is what actually gets transmitted.

Figure 5.13 *Division in CRC encoder*



Modulo-2 (binary/XOR) division used by the CRC encoder to compute the remainder — the CRC bits.

Worked example set-up: a 4-bit dataword, using a 4-bit divisor (1011), needs 3 appended zeros (since the divisor has 4 bits, it will produce a 3-bit remainder). This gives a 7-bit codeword overall (4-bit dataword + 3-bit CRC).

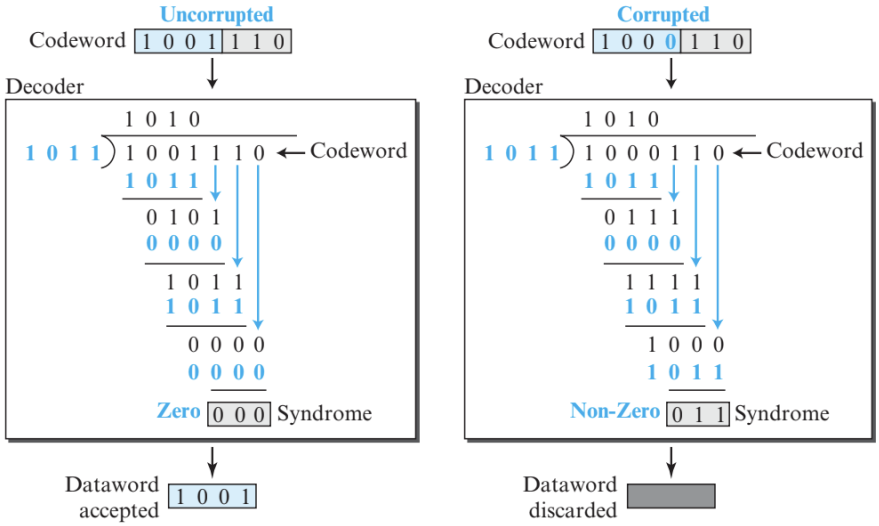
Modulo-2 Division

- Addition and subtraction in modulo-2 division are both performed using the XOR operation (no borrowing or carrying is involved).
- The quotient produced by the division is simply ignored.
- Only the remainder of the division is retained and used, as the CRC value.
- At each step of long division, if the leftmost bit of the current partial dividend is 0, an all-zero pattern (rather than the actual divisor) is XORed in for that step.
- The remainder left over after the very last division step becomes the final CRC check bits.

CRC Decoder (Checking for Errors)

At the receiver, the entire received codeword (dataword bits plus CRC bits, exactly as received, possibly corrupted) is divided by the very same generator (divisor) that the sender used, again using modulo-2 division. The remainder produced by this division is called the syndrome.

Figure 5.14 *Division in the CRC decoder for two cases*



CRC decoder: the received codeword is divided by the same generator; a zero syndrome means the frame is accepted.

- Syndrome = 000...0 (all zeros): no detectable error — the dataword is accepted and delivered upward.
- Syndrome ≠ 000...0: an error has been detected — the received frame is discarded.

Thus, CRC allows the receiver to determine, with very high confidence, whether the received data is error-free before it is handed up to the next layer, without the receiver ever needing to know the original dataword in advance.

The Generator (Divisor)

The generator (divisor) is a fixed binary bit pattern, agreed upon in advance by both the sender and the receiver — typically, it is a standard defined by the relevant networking specification rather than something chosen arbitrarily. Its required length is:

`Generator length = (n − k + 1) bits`

Widely used standard generators include CRC-8, CRC-16, and CRC-32. The number that appears in each name refers to the degree of the generator polynomial (i.e., the number of CRC bits it produces), so:

- CRC-8 uses a 9-bit generator (produces 8 CRC bits).
- CRC-16 uses a 17-bit generator (produces 16 CRC bits).
- CRC-32 uses a 33-bit generator (produces 32 CRC bits) — this is the variant used by Ethernet, as covered later in this module.

Requirements of a Valid CRC Generator

For a given bit pattern to be usable as a CRC generator (divisor), it must satisfy two conditions:

- The generator must contain at least two bits.
- Both the first (leftmost) bit and the last (rightmost) bit of the generator must be 1.

These two conditions are what guarantee the generator's ability to reliably detect transmission errors — a generator that violates them would fail to detect certain common error patterns.

Performance of CRC

- Single-bit errors: a valid CRC generator (satisfying the two conditions above) is guaranteed to detect every possible single-bit error.
- Odd number of errors: a valid generator detects all odd numbers of bit errors, provided it is exactly divisible by the pattern $(11)_2$ under modulo-2 division; otherwise, it may only catch some odd-numbered error patterns.

For burst errors, let L be the length of the burst error and r be the number of CRC (remainder) bits used:

- If $L \leq r$: every burst error of this length is detected, with 100% certainty.
- If $L = r + 1$: burst errors of this length are detected with probability $1 - (0.5)^{(r-1)}$.
- If $L > r + 1$: burst errors of this length are detected with probability $1 - (0.5)^{r}$.

Because these detection probabilities approach 100% extremely quickly as r grows (even r = 16 or r = 32, as used by CRC-16/CRC-32, gives detection probabilities indistinguishable from certainty for practical purposes), CRC is considered one of the most reliable and efficient error-detection techniques in use, and is the technique of choice in Ethernet, LANs, WANs, USB, and storage devices.

Advantages of Cyclic Codes (CRC) — Summary

- Easy to implement in both hardware (shift registers with XOR gates) and software.
- Very fast, especially in a hardware implementation, since it can be computed 'on the fly' as bits arrive.
- Detects single-bit, double-bit, all odd-bit, and (with near certainty) burst errors.
- Extremely widely used — Ethernet, LANs, WANs, USB, storage devices and many other communication protocols all rely on CRC.

8. Checksum

Checksum is another well-known error-detection technique used to verify data integrity. Unlike CRC, which is primarily a Data Link Layer technique, checksum is mainly used at the Network Layer and Transport Layer — for example, within the headers of IP, TCP, and UDP.

8.1 Working of Checksum

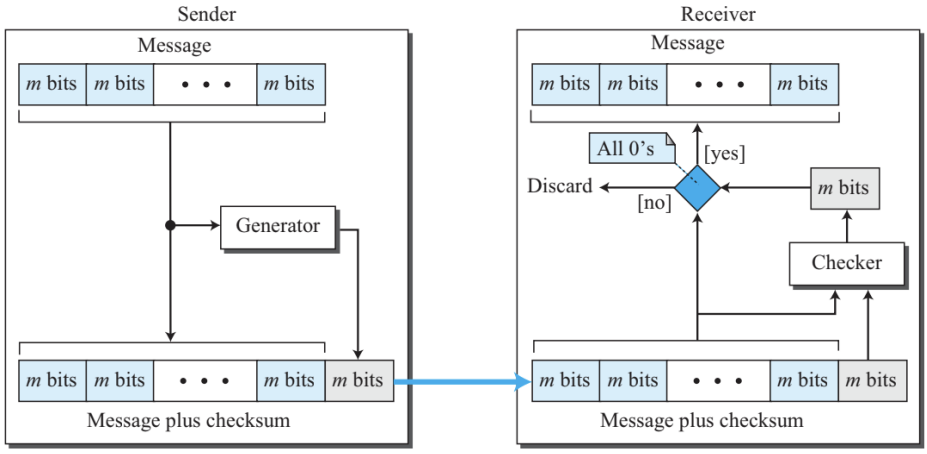
At the sender:

- The message is divided into a number of m-bit units.
- All of these m-bit units are added together (using one's-complement arithmetic, described below).
- The result is used to generate an m-bit checksum value.
- The message is transmitted together with this checksum value appended to it.

At the receiver:

- The received message is again divided into m-bit units.
- All of the received message's units and the received checksum value are added together.
- This produces a new checksum.
- Decision: if the new checksum is all 0s, no error is detected and the message is accepted; if the new checksum is anything other than all 0s, an error is detected and the message is discarded.

Figure 5.15 *Checksum*



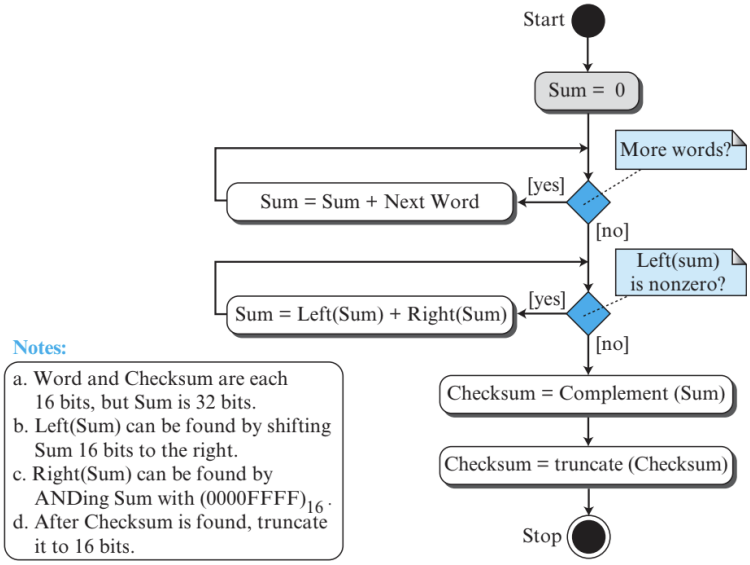
Checksum at sender and receiver — the message is divided into units, summed, and complemented to produce the checksum.

The checksum field's exact position within the overall message depends on the specific protocol using it (for example, it sits within the fixed header for TCP/UDP/IP), but the underlying add-and-complement principle remains the same.

8.2 Checksum Algorithm (One's-Complement Sum)

- Initialize Sum = 0.
- Divide the message into 16-bit words.
- Add each word into Sum using two's-complement addition.
- If an overflow (carry-out of the most significant bit) occurs during any addition, wrap that carry around and add it back into the least significant bit position — this simulates one's-complement arithmetic using ordinary adder hardware.
- Repeat until every word of the message has been added.
- Take the one's complement (bitwise NOT) of the final accumulated sum.
- The result is the 16-bit checksum, which is transmitted along with the message.

Figure 5.17 *Algorithm to calculate a traditional checksum*



Steps of the checksum algorithm — one's-complement addition with end-around carry, followed by complementing.

At the receiver, the identical calculation is repeated over the received message together with the received checksum value. If the result is all 0s, no error is detected and the message is accepted; any other result means an error has been detected and the message must be discarded.

8.3 Performance and Limitations of Checksum

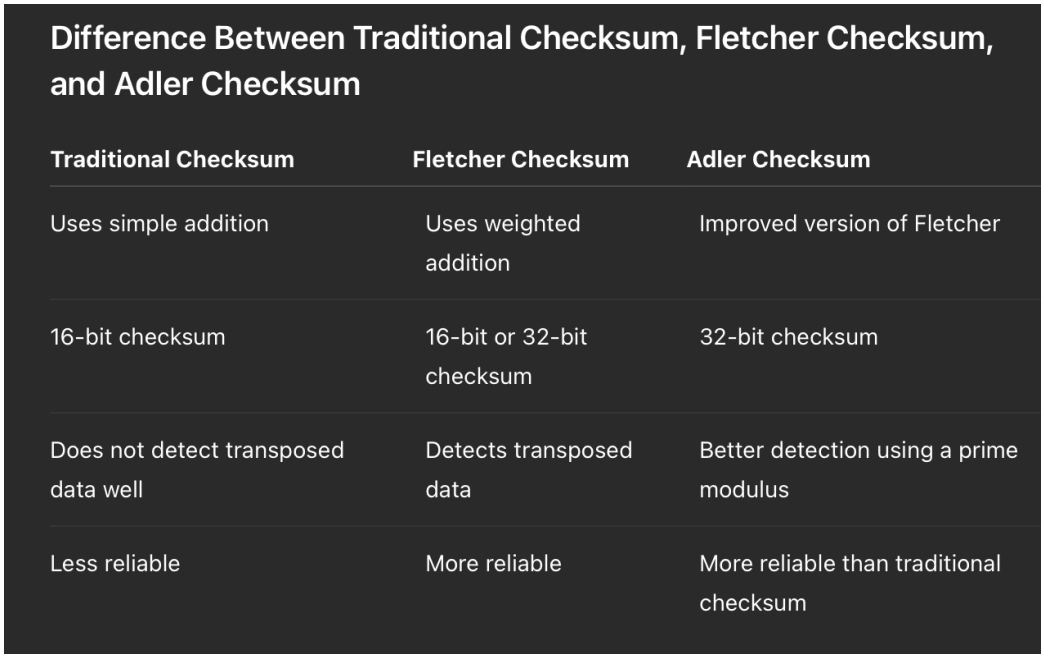
The traditional (Internet) checksum uses only 16 bits to protect messages that may run to thousands of bits, so although it is computationally cheap and simple, it is considerably less reliable at catching errors than CRC.

- If one data word is increased by a certain amount and another data word is simultaneously decreased by the exact same amount, the overall sum — and hence the checksum — remains unchanged, so the error goes undetected.
- If two data words are transposed (swapped in position) but their values are otherwise unchanged, the checksum also remains unchanged, because plain addition does not take word order into account.

Because of these blind spots, modern communication protocols increasingly favour CRC over the traditional Internet checksum wherever feasible.

8.4 Fletcher Checksum

The Fletcher Checksum is an improved checksum algorithm designed specifically to overcome the position-blindness of the traditional checksum. It assigns a position-dependent weight to each data item, so that transposed (swapped) data — which the traditional checksum cannot detect — is now caught, because swapping two items changes their weighted contributions differently.



Adler checksum, an improvement on the Fletcher checksum, using a prime modulus for stronger error detection.

- 8-bit Fletcher Checksum: operates on 8-bit (1-byte) data items, produces a 16-bit checksum, and performs its running calculations modulo 256 (2^8).
- 16-bit Fletcher Checksum: operates on 16-bit data items, produces a 32-bit checksum, and performs its running calculations modulo 65,536 (2^{16}).

The algorithm maintains two running accumulators: L, which stores the plain running sum of the data, and R, which stores a running weighted sum (where each new value of L is itself added into R, so items further into the message accumulate more weight). This weighting is precisely what makes the Fletcher checksum considerably more reliable than the traditional checksum at detecting reordering and certain patterns of corruption.

8.5 Adler Checksum

The Adler Checksum is a further refinement, based directly on the Fletcher checksum idea but tuned for slightly stronger error detection:

- Produces a 32-bit checksum.
- Processes data one byte (8 bits) at a time.
- Uses a prime modulus of 65,521 (instead of the power-of-two modulus 65,536 used by the plain Fletcher checksum).
- Initializes the running accumulator L to 1, instead of 0.

Using a prime number as the modulus is a deliberate design choice that improves error-detection performance for certain classes of data patterns that would otherwise weaken a power-of-two modulus.

9. Data Link Control (DLC) Protocols

Having covered the general functions of the DLC sublayer — framing, flow control, and error control — we now look at two concrete, widely deployed DLC protocols that put these functions into practice: High-Level Data Link Control (HDLC) and the Point-to-Point Protocol (PPP).

10. High-Level Data Link Control (HDLC)

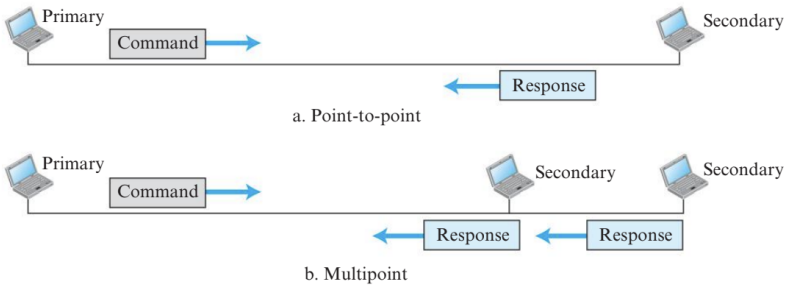
HDLC is a bit-oriented Data Link Layer protocol used for reliable communication over both point-to-point and multipoint links. It provides framing, flow control, and error control, traditionally implemented using the Stop-and-Wait ARQ family of protocols. Although in modern networks PPP (built on HDLC-like framing) is more commonly seen at the edge, HDLC's frame structure and terminology form the conceptual basis for many later protocols, which is why it remains important to study.

10.1 HDLC Transfer (Configuration) Modes

1. Normal Response Mode (NRM)

- Uses an unbalanced configuration: one primary station and one or more secondary stations.
- Only the primary station is allowed to initiate communication by sending commands; secondary stations may only respond when spoken to.
- Used in both point-to-point and multipoint network arrangements.

Figure 5.20 *Normal response mode*



In ABM, the configuration is balanced. The link is point-to-point, and each station can function as a primary and a secondary (acting as peers), as shown in Figure 5.21. This is the common mode today.

Figure 5.21 *Asynchronous balanced mode*



Normal Response Mode (NRM): the primary issues a command, and secondary station(s) respond — shown for both point-to-point and multipoint configurations.

2. Asynchronous Balanced Mode (ABM)

- Uses a balanced configuration: both stations have equal (peer) status.
- Either station may transmit data at any time, without first waiting for permission from the other.

- Used only over point-to-point links.
- This is the mode most commonly used in practice today, since most links are simple point-to-point connections between two peers.

10.2 HDLC Frame Types

HDLC defines exactly three kinds of frames, distinguished by the leading bit(s) of the Control field:

1. Information Frame (I-Frame)

- Carries actual user data.
- Also carries flow-control and acknowledgment information, using a technique called piggybacking (embedding the ACK for received data inside an outgoing data frame, rather than sending a separate ACK frame).
- Used for ordinary, everyday data transmission.

2. Supervisory Frame (S-Frame)

- Used purely for flow control and error control purposes.
- Carries no user data at all.
- Used specifically in situations where piggybacking is not possible — for example, when a station has no outgoing data of its own on which to piggyback an acknowledgment.

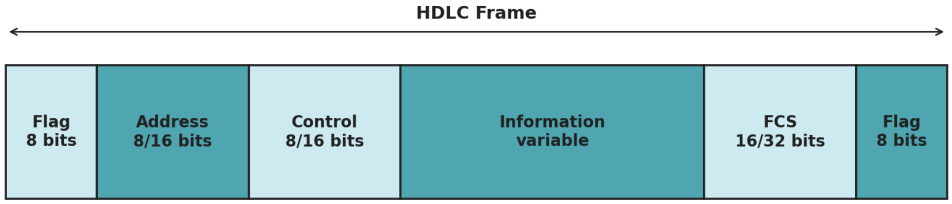
3. Unnumbered Frame (U-Frame)

- Used for system management and link management functions (such as setting up or tearing down a connection).
- Carries control/management information rather than user data.

10.3 HDLC Frame Format

Every HDLC frame is built from the following fields, in order:

- Flag (8 bits) — marks the beginning of the frame; the fixed bit pattern 01111110.
- Address (8 or 16 bits) — the address of the secondary station (for NRM) or the relevant peer.
- Control (8 or 16 bits) — identifies the frame type (I, S, or U) and carries flow-/error-control information.
- Information — carries user data (present in I-frames; may be absent for some S- and U-frames); this field has a variable, unrestricted length.
- Frame Check Sequence (FCS) (16 or 32 bits) — a CRC value used for error detection.
- Flag (8 bits) — marks the end of the frame; again the pattern 01111110.

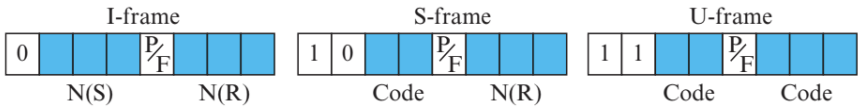


General HDLC frame format: Flag – Address – Control – Information – FCS – Flag.

Control Field in HDLC

The exact interpretation of the Control field's bits depends on which of the three frame types is in use:

Figure 5.23 *Control field format for the different frame types*



Control field format for the three HDLC frame types — I-frame, S-frame, and U-frame.

I-Frame Control Field

- The very first bit of the Control field is always 0, which is how the receiver recognises it as an I-frame.
- N(S) — a sequence number, identifying this frame within the ongoing stream of data frames.
- N(R) — an acknowledgment (piggybacked) number, indicating the next frame number expected from the peer.
- P/F bit — the Poll/Final bit, whose meaning depends on the direction of travel: when sent from the Primary to a Secondary, it is interpreted as a Poll (asking the secondary to respond); when sent from a Secondary to the Primary, it is interpreted as Final (indicating the last frame of the secondary's current response).

S-Frame Control Field

The first two bits of the Control field are 10, marking it as an S-frame. S-frames come in four subtypes, called codes:

- Receive Ready (RR) — a positive acknowledgment (ACK); tells the sender that the receiver is ready for more frames.
- Receive Not Ready (RNR) — acknowledges frames received so far, but also signals that the receiver is currently busy and cannot accept any additional frames for the moment.
- Reject (REJ) — a negative acknowledgment (NAK), used with Go-Back-N ARQ; asks the sender to retransmit starting from the damaged or lost frame onward.
- Selective Reject (SREJ) — a negative acknowledgment used with Selective-Repeat ARQ; asks the sender to retransmit only the one specific erroneous frame, not everything after it.

U-Frame Control Field

The first two bits of the Control field are 11, marking it as a U-frame. U-frames are used purely for link and session management (such as link setup, mode-setting, and disconnection), rather than for carrying or acknowledging user data, and HDLC supports up to 32 distinct U-frame management commands.

11. Point-to-Point Protocol (PPP)

The Point-to-Point Protocol (PPP) is a Data Link Layer protocol specifically designed for communication over a single point-to-point link. It is very widely used to connect an end-user's computer to an Internet Service Provider (ISP) — historically over dial-up telephone lines, and today over leased lines, DSL, and similar direct connections.

11.1 Services Provided by PPP

- Defines a clear frame format for the data being exchanged.
- Handles establishing, maintaining, and eventually terminating the communication link.
- Supports multiple different Network Layer protocols simultaneously (for example, IP as well as IPv6) through a multiplexing scheme.
- Provides optional user authentication, to verify the identity of the party at the other end of the link before allowing access.
- Supports Multilink PPP (MLPPP), which allows several separate physical links to be bonded together and used as though they were a single logical connection, for extra bandwidth.
- Provides dynamic network-address configuration, allowing IP-layer parameters to be negotiated automatically when the link comes up.

11.2 Services NOT Provided by PPP

To keep the protocol deliberately simple and lightweight, PPP does not provide:

- Flow control.
- Sequence numbering of frames.
- Advanced (correcting) error control — PPP only detects errors, using CRC.
- Multipoint addressing — PPP is strictly for point-to-point links only, with exactly two parties.

Because PPP performs only error detection (via CRC) and not error correction, any frame found to contain an error is simply discarded outright; it is then the responsibility of an upper-layer protocol (such as TCP, at the Transport Layer) to notice the missing data and arrange for retransmission.

11.3 PPP Frame Format and Byte Stuffing

PPP is a byte-oriented protocol (unlike bit-oriented HDLC), and it therefore uses the same flag-byte idea seen earlier in character-oriented framing, together with byte stuffing to keep the flag pattern unambiguous:

- If the flag byte pattern (01111110) happens to occur naturally within the payload being carried, it could be mistaken by the receiver for the end of the frame.

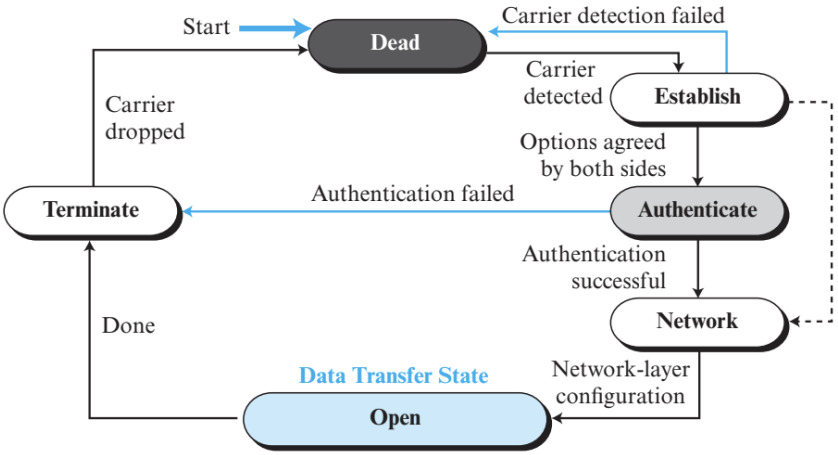
- To prevent this, PPP uses Byte Stuffing: a special Escape Byte (01111101) is inserted immediately before every occurrence of the flag byte pattern found within the data.
- If the Escape Byte itself happens to occur within the data, an additional Escape Byte is inserted before it, exactly as in ordinary character-oriented byte stuffing.
- At the receiver, these inserted escape bytes are detected and removed, restoring the data to its original, unstuffed form.

11.4 PPP Transition Phases

Over the lifetime of a single connection, PPP moves through a well-defined sequence of states, from the moment the link is first activated to the moment it is finally torn down:

- Dead — the initial (and final) state: there is no active physical-layer carrier signal, and the line is completely inactive.
- Establish — once one of the two devices starts communication and a carrier signal is detected, the connection enters the Establish state. Here, the two devices negotiate and agree on the connection options to be used, using the Link Control Protocol (LCP).
- Authenticate — if authentication has been configured as a requirement, the devices now perform it. A successful authentication moves the connection on to the Network state; a failed authentication instead moves the connection straight to Terminate.
- Network — Network-Layer parameters (such as IP addresses) are negotiated and configured, using the appropriate Network Control Protocol (NCP) for whichever upper-layer protocol is being carried.
- Open (Data Transfer State) — once configuration succeeds, the connection enters the Open state, and genuine data transfer between the two devices' Network Layers takes place here.
- Terminate — when either device wishes to end the connection, it enters the Terminate state, where it remains until the underlying physical-layer carrier is actually dropped.
- Dead — once the carrier is dropped, the connection returns to the initial Dead state, completing the cycle.

Figure 5.25 *Transition phases*



PPP transition phases: Dead → Establish → Authenticate → Network → Open → Terminate → Dead.

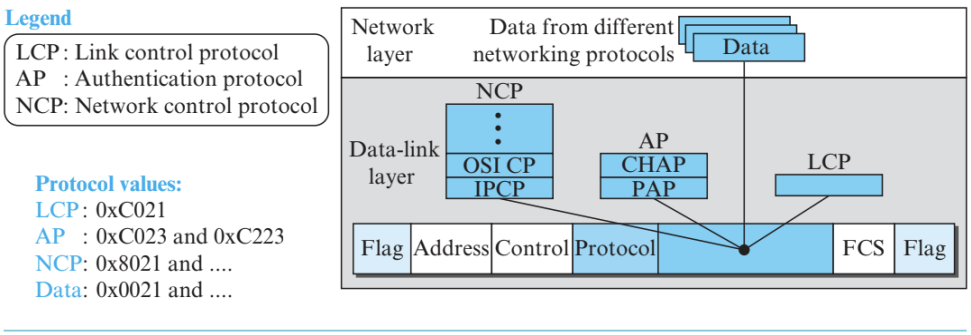
11.5 PPP Multiplexing

Even though PPP itself operates at the Data Link Layer, it relies internally on several other, smaller protocols to carry out its different jobs, all multiplexed over the same underlying PPP connection so that a single PPP link can carry many different kinds of control and data traffic:

- Link Control Protocol (LCP) — establishes, configures, and manages the PPP link itself.
- Authentication Protocols (APs) — authenticate the identity of the devices at either end of the link.
- Network Control Protocols (NCPs) — configure and manage the specific Network-Layer protocol(s) that will actually ride on top of the PPP link.

A single PPP packet's payload may therefore carry data belonging to LCP, to an authentication protocol, to an NCP, or to an actual Network-Layer protocol — the receiver uses the Protocol field (described below) to know which of these it is dealing with.

Figure 5.26 *Multiplexing in PPP*



PPP protocol stack: LCP, authentication protocols (PAP/CHAP), and NCPs all share the same PPP link.

Link Control Protocol (LCP)

LCP is responsible for establishing, maintaining, configuring, and eventually terminating a PPP link. It also negotiates various connection options between the two endpoints — both sides must reach agreement on the required options before the underlying link can actually be brought up and used for data transfer.

Authentication Protocols

Authentication is the process of verifying the identity of a user or device before permitting it access to network resources over the link. PPP defines two authentication protocols:

- Password Authentication Protocol (PAP) — a simple two-step scheme in which the user sends a username and password to the system, which checks them and either accepts or rejects the connection. PAP is simple to implement, but relatively insecure, because the password is transmitted across the link (largely) in the clear.
- Challenge Handshake Authentication Protocol (CHAP) — a three-way handshake that is considerably more secure than PAP, because the actual password is never transmitted directly across the link at all.

CHAP works as follows: the system first sends a challenge value to the user; the user combines this challenge value with their password using a predefined one-way function, and sends only the resulting computed value back; the system then performs the exact same computation itself, using its own stored copy of the password, and compares the two results — access is granted only if they match, and denied otherwise. Because the challenge value can be changed on every attempt, and the actual password is never transmitted, CHAP offers substantially stronger protection than PAP.

Network Control Protocols (NCPs)

Since PPP is capable of carrying traffic belonging to different Network-Layer protocols, it uses a distinct Network Control Protocol (NCP) to configure the link appropriately for each one. For example, IPCP (Internet Protocol Control Protocol) configures PPP specifically to carry IP packets; other NCPs exist for other Network-Layer protocols. It is important to note that NCP packets themselves never carry actual network-layer user data — their sole job is to configure the link so that it is ready to carry that data afterward.

Once the relevant NCP has finished its configuration, genuine Network-Layer data can finally be exchanged over the link. The PPP frame's Protocol field identifies exactly what kind of payload is being carried in a given frame — for instance, IP data is marked with Protocol field value 0021_{16}, while OSI Network-Layer data is marked with Protocol field value 0023_{16}.

11.6 Multilink PPP (MLPPP)

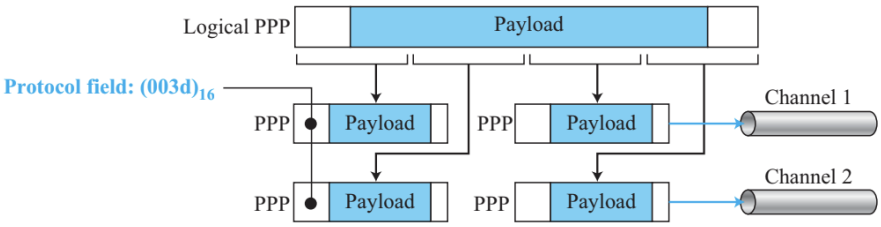
PPP was originally designed with the assumption of a single physical channel connecting two devices. As it became common for two devices to be connected by more than one physical channel simultaneously (for extra bandwidth or redundancy), Multilink PPP (MLPPP) was developed to make use of them together, as one combined logical connection.

- A single logical PPP frame is broken up into several smaller fragments.
- Each fragment is carried inside its own separate, ordinary PPP frame.

- The Protocol field of each such carrier frame is set to the special value $003D_{16}$, signalling that this frame actually carries a fragment of some larger logical PPP frame, rather than a complete frame in its own right.
- A sequence number is included with each fragment, so the receiver knows the correct order in which to reassemble them.
- At the receiving end, all the fragments belonging to one logical frame are collected and reassembled into the original, complete logical PPP frame.

Simple flow: Logical PPP Frame → Fragmentation → Multiple (smaller) PPP Frames sent, potentially over multiple physical links in parallel → Transmission → Reassembly at the far end → Original logical frame recovered.

Figure 5.27 *Multilink PPP*



Multilink PPP: a single logical frame is fragmented, sent as several PPP frames (possibly over different physical links), and reassembled at the receiver.

12. Multiple Access Protocols (the MAC Sublayer)

The Media Access Control (MAC) sublayer is responsible for controlling how multiple devices share one common communication medium. While the DLC sublayer manages reliable data transfer across a single link between two nodes, the MAC sublayer is specifically concerned with managing contention for access to a shared channel among many nodes at once.

When a dedicated (point-to-point) link is in use — such as a dial-up telephone line — only a DLC protocol like PPP is required, since exactly two devices communicate and there is no possibility of a third party interfering. However, when several devices must share the very same transmission medium — as in a wired LAN using a shared bus/hub, or any wireless network sharing the same radio channel — a Multiple Access Protocol is essential to coordinate access to that shared medium.

A shared communication channel of this kind is called a multipoint link or broadcast link. In such a network, several devices may attempt to transmit at the same time; without proper coordination, two or more simultaneous transmissions interfere with each other, resulting in a collision in which the transmitted frames become garbled.

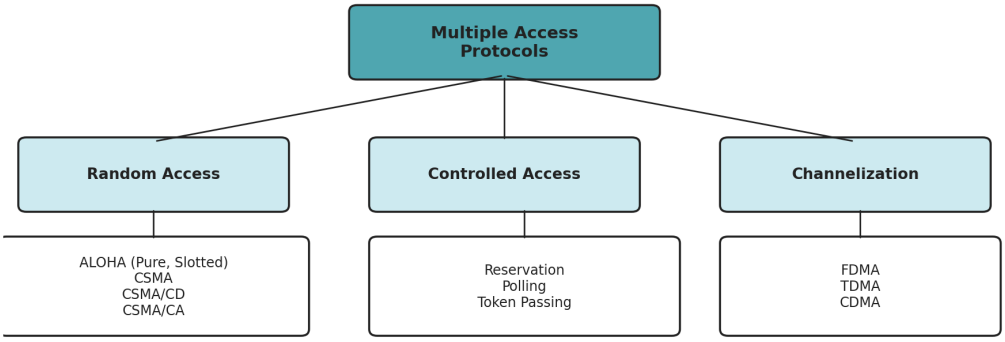
Objectives of Multiple Access Protocols

- Coordinate access to the shared communication medium among all competing stations.
- Prevent collisions between simultaneous transmissions wherever possible.

- Detect collisions when they do occur, and recover from them (for example, through retransmission).
- Ensure fair access, so that every device eventually gets an opportunity to transmit.
- Avoid any single device from monopolizing the communication channel indefinitely.

Note: The operation of a multiple-access protocol is analogous to the informal rules followed during a group discussion or meeting, where ideally only one person speaks at a time while everyone else listens and waits their turn.

12.1 Classification of Multiple Access Protocols



Classification of multiple access protocols into Random Access, Controlled Access, and Channelization.

- Random Access Protocols — devices transmit whenever they have data ready to send; collisions may occur and are dealt with only after they happen. Examples: ALOHA, Slotted ALOHA, CSMA, CSMA/CD, and CSMA/CA.
- Controlled Access Protocols — devices are only permitted to transmit after receiving explicit permission from some coordinating mechanism; collisions are avoided proactively through this controlled access. Examples: Reservation, Polling, and Token Passing.
- Channelization Protocols — the communication channel's total capacity is divided up among the multiple users in advance, with each user receiving a dedicated portion of that capacity (in frequency, time, or code). Examples: FDMA (Frequency Division Multiple Access), TDMA (Time Division Multiple Access), and CDMA (Code Division Multiple Access).

13. Random Access Protocols

In random access protocols, all stations have equal, symmetric access rights to the shared medium; there is no central controller granting or denying permission to transmit. Consequently, each station individually follows a shared set of rules to try to avoid — or, failing that, to resolve — access conflicts on its own.

Before transmitting, each station effectively needs an answer to the following questions:

- When can the station legitimately access the medium?

- What should the station do if it finds the medium is currently busy?
- How does the station determine, after the fact, whether its transmission succeeded or failed?
- What should the station do next if a collision (access conflict) does occur?

Random-access methods have evolved historically as follows, each addressing a shortcoming of the previous generation:

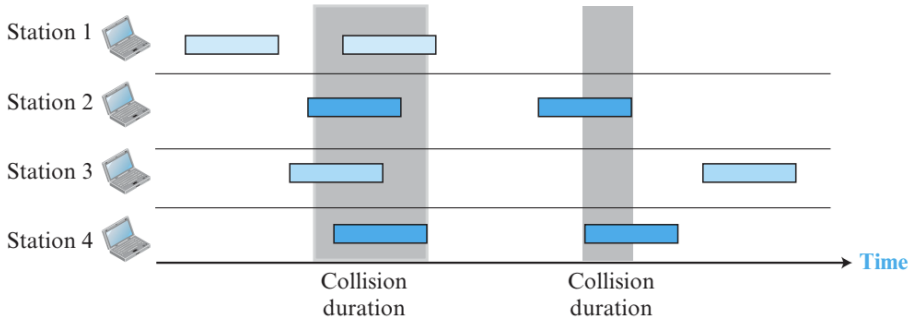
- ALOHA (pure Multiple Access, MA) — stations simply transmit whenever they have data, with no attempt to check the medium first.
- CSMA (Carrier Sense Multiple Access) — stations first sense (listen to) the medium before transmitting.
- CSMA/CD (CSMA with Collision Detection) — additionally detects collisions while they are happening (during transmission) and aborts/retransmits.
- CSMA/CA (CSMA with Collision Avoidance) — actively attempts to avoid collisions before they happen in the first place, rather than merely detecting them afterward.

13.1 ALOHA

ALOHA is the earliest random-access protocol, developed at the University of Hawaii in the early 1970s originally for a wireless (radio) LAN connecting the Hawaiian islands, though the same idea applies to any shared communication medium. Because multiple stations share the same channel, collisions can occur whenever two or more stations transmit at overlapping times; even a single overlapping bit between two frames is enough to corrupt both of them, requiring both to be retransmitted.

Pure ALOHA

Figure 5.29 *Frames in a pure ALOHA network*



Frames in Pure ALOHA: stations transmit at arbitrary times, so a collision destroys any frames that overlap even partially.

Working of Pure ALOHA:

- A station transmits a frame immediately, the instant it has data ready to send, with no coordination whatsoever.
- The receiver sends back an Acknowledgment (ACK) if the frame was received correctly.

- If the sender fails to receive this ACK before a timeout period elapses, it assumes the frame was lost, most likely due to a collision.
- The sender then waits for a random back-off time (T^B) before attempting to retransmit the same frame.
- This randomized waiting period is crucial: it reduces the chance that the same two stations collide again on their very next retry.
- If, despite repeated attempts, the frame still cannot be transmitted successfully after a maximum number of retransmission attempts (K_{max}), the station gives up retransmitting for now and tries again only later.

Characteristics, Advantages, and Disadvantages of Pure ALOHA

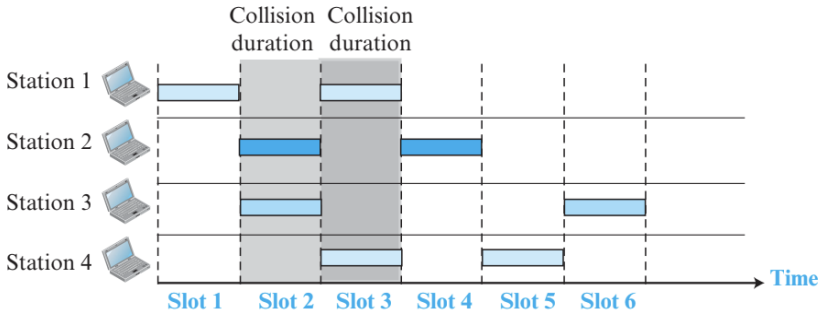
Advantages	**Disadvantages**
Very simple protocol to understand and implement	High probability of collisions, since there is no coordination at all
Well suited to small or lightly loaded networks	Low overall channel efficiency, especially under heavy load
No need for any time synchronization between stations	Frequent retransmissions increase overall network delay
	Performance degrades badly under heavy traffic

Slotted ALOHA

Slotted ALOHA is a straightforward but effective improvement on Pure ALOHA, designed to reduce collisions and increase overall network efficiency. In Pure ALOHA, a station can begin transmitting at any arbitrary instant, giving it a vulnerable time of $2 \times T^f_r$ (where T^f_r is the time taken to transmit one frame) — any other station starting anywhere within that $2 \times T^f_r$ window will cause a collision.

To reduce this, Slotted ALOHA divides time into discrete, equal-length slots, each exactly T^f_r seconds long, and requires every station to begin transmission only at the start of a slot — never at an arbitrary mid-slot instant.

Figure 5.32 *Frames in a slotted ALOHA network*

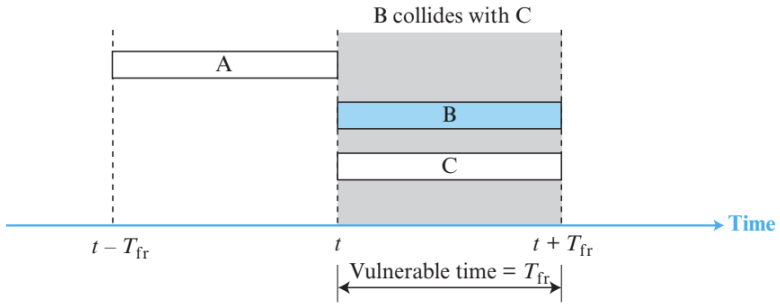


In Slotted ALOHA, time is divided into slots, and stations may only begin transmitting at the start of a slot.

If a station has data ready but happens to miss the start of the current slot, it must simply wait until the beginning of the next slot before transmitting. Because every transmission now always begins precisely at

a slot boundary, the vulnerable time shrinks to just T^f_r — exactly half of what it was under Pure ALOHA. A collision can, however, still occur if two or more stations both happen to begin transmitting at the start of the very same slot.

Figure 5.33 *Vulnerable time for slotted ALOHA protocol*



Slotted ALOHA vulnerable time = T_{fr}

Vulnerable time comparison: Pure ALOHA (2 x Tfr) versus Slotted ALOHA (Tfr).

As with Pure ALOHA, the sender waits for an Acknowledgment (ACK) after transmitting; if none arrives before the timeout, a collision is assumed, and the sender waits a random back-off period before retrying, up to a maximum of K_{max} attempts, after which it gives up for the time being.

Throughput of Slotted ALOHA

The throughput S of Slotted ALOHA (the fraction of transmitted frames that succeed, as a function of offered load) is given by:

$$S = G \times e^{-G}$$

where S is the throughput and G is the average number of transmission attempts per frame time (the offered load). Differentiating this expression shows that the maximum throughput occurs precisely when G = 1, giving:

$$S_{max} = 0.368\ (36.8\%),\ \text{achieved at } G = 1$$

In other words, when — on average — exactly one frame's worth of transmission attempts occurs during each frame time, roughly 36.8% of all transmitted frames are successfully delivered without collision. (By comparison, Pure ALOHA's maximum throughput works out to only 18.4%, exactly half of Slotted ALOHA's — a direct consequence of the halved vulnerable time.)

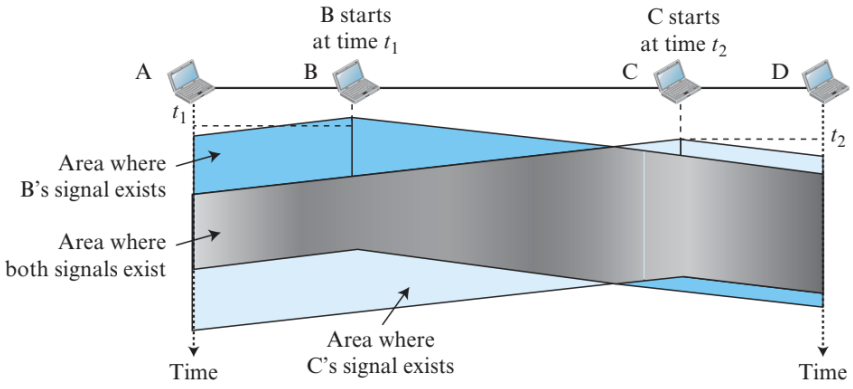
- Advantages of Slotted ALOHA: fewer collisions than Pure ALOHA; vulnerable time reduced from $2 \times T^f_r$ to just T^f_r; higher maximum throughput (36.8% versus 18.4%); better overall channel utilization.
- Disadvantages of Slotted ALOHA: requires all stations to maintain accurate time synchronization; collisions can still occur when multiple stations start transmitting at the same slot boundary; a station may need to wait for the next slot even when the channel is otherwise idle, adding some delay.

14. Carrier Sense Multiple Access (CSMA)

CSMA is a random-access protocol developed to reduce collisions and improve on ALOHA's performance. Unlike ALOHA, before transmitting, a CSMA station first listens to (senses) the communication medium — this is why CSMA is often summarised by the phrase 'Sense Before Transmit' or 'Listen Before Talk.'

are connected to a shared channel (usually a dedicated medium).

Figure 5.34 *Space/time model of a collision in CSMA*



In CSMA, a station listens to the channel before transmitting; here, stations B, C, and D each sense the medium at different times.

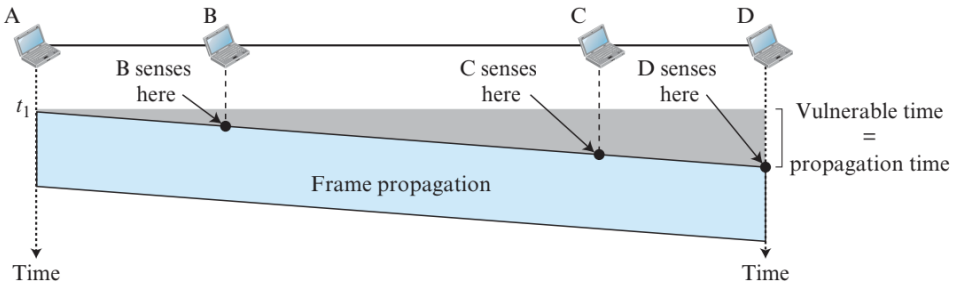
If the channel is found to be idle, the station attempts to transmit. If the channel is found to be busy, the station instead waits, following one of several persistence methods (described below), before trying again.

Although sensing the channel first significantly reduces collisions compared with ALOHA, CSMA still cannot eliminate collisions completely. This is a direct consequence of propagation delay (T_p): when one station begins transmitting, its signal takes a small but nonzero amount of time to physically reach every other station on the medium. During this brief propagation window, another station elsewhere on the medium may still sense the channel as idle (because the first station's signal simply hasn't reached it yet) and may itself begin transmitting — causing a collision.

14.1 Vulnerable Time in CSMA

The vulnerable time in CSMA is equal to the propagation time, T_p — the time needed for a signal to travel from one end of the shared medium to the other end.

Figure 5.35 *Vulnerable time in CSMA*



Vulnerable time in CSMA: any station that starts transmitting within Tp of the first transmission causes a collision.

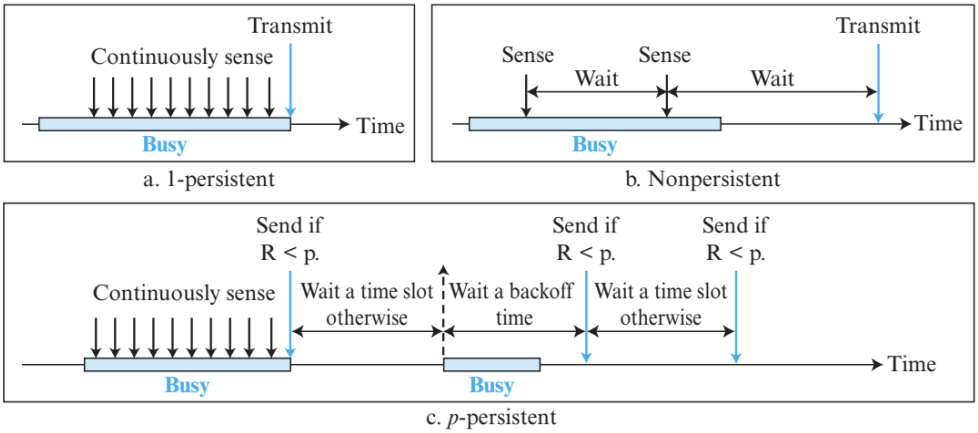
$$\texttt{Vulnerable Time} = T_p$$

If some other station begins transmitting at any point during this propagation window T_p, a collision results. However, once the very first bit of the leading station's signal has physically reached every other station on the medium, all of those stations will now correctly sense the medium as busy and will refrain from transmitting — so the danger window closes as soon as propagation is complete.

14.2 Persistence Methods in CSMA

Persistence methods precisely define how a station should behave depending on whether it currently finds the channel busy or idle. Three persistence methods are commonly used:

Figure 5.36 *Behavior of three persistence methods*



Behaviour of the three CSMA persistence methods: 1-persistent, nonpersistent, and p-persistent.

1. 1-Persistent CSMA

- The station continuously senses the channel.
- If the channel is idle, the station transmits immediately, with probability 1 (hence the name '1-persistent').

- If the channel is busy, the station keeps sensing the channel continuously until it becomes idle, and then transmits immediately at that moment.
- Advantage: simple to implement, and there is no unnecessary waiting once the channel becomes free.
- Disadvantage: a relatively high probability of collisions, since every station that was waiting for the channel to free up will all try to transmit at essentially the same instant the channel becomes idle.
- This method is used in traditional (shared-medium) Ethernet, as part of CSMA/CD.

2. Nonpersistent CSMA

- The station senses the channel.
- If the channel is idle, it transmits immediately.
- If the channel is busy, the station does not keep monitoring continuously; instead, it waits for a random period of time before sensing the channel again.
- Advantage: reduces the probability of collisions compared with 1-persistent CSMA, because competing stations are unlikely to all pick the exact same random moment to re-check the channel.
- Disadvantage: lower channel utilization, because the medium may sit idle for a while even though a waiting station has data ready, simply because that station hasn't finished its random wait yet.

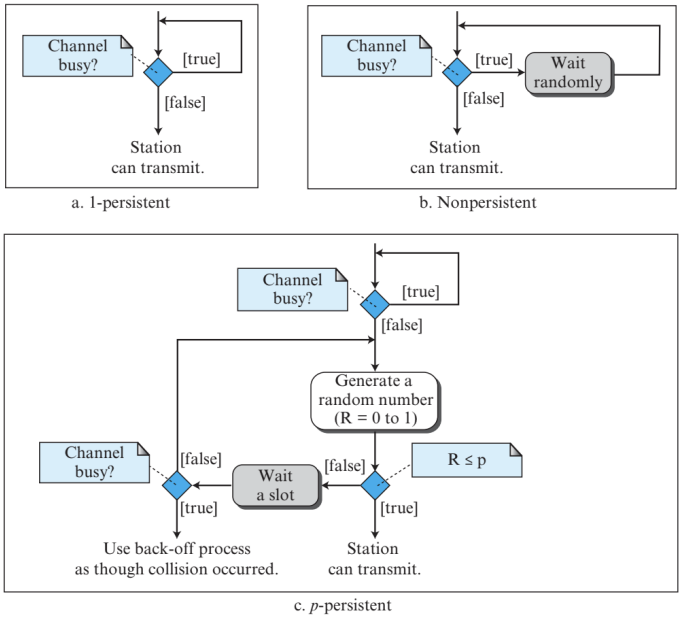
3. p-Persistent CSMA

- Used specifically on slotted channels, where the slot duration is chosen to be equal to or greater than the maximum propagation time.
- When the channel becomes idle, the station transmits with a fixed probability p, or instead defers to the next time slot with probability $q = 1 - p$.
- At the start of the next slot, if the channel is still idle, the exact same probabilistic process is repeated; if the channel has since become busy, the station performs the appropriate back-off procedure before trying again.
- Advantage: reduces collisions relative to 1-persistent CSMA, while also improving channel utilization relative to nonpersistent CSMA — providing a tunable middle ground between the other two extremes, controlled by the choice of p.

15. CSMA/CD (Collision Detection)

Carrier Sense Multiple Access with Collision Detection (CSMA/CD) improves further on plain CSMA by adding a procedure for actively detecting a collision while it is happening, and reacting to it immediately, rather than simply waiting for a timeout as in ALOHA.

Figure 5.37 *Flow diagram for three persistence methods*



CSMA/CD: stations continue monitoring the channel throughout their own transmission, so a collision can be detected almost immediately.

In CSMA/CD, a station first follows the ordinary CSMA persistence procedure and begins transmitting once the channel is judged available. Crucially, while transmitting, the station continues to actively monitor the medium at the same time. If the transmission completes without any sign of collision, the station simply finishes normally. If, however, a collision is detected mid-transmission, the station immediately aborts (stops) its transmission and schedules a retransmission later, using a back-off procedure.

15.1 Working of CSMA/CD

- The station senses the channel.
- If the channel is busy, it follows the appropriate persistence method (as in ordinary CSMA).
- If the channel is idle, the station starts transmitting.
- While transmitting, the station continuously monitors the channel for signs of a collision.
- If no collision is detected, the transmission completes successfully.
- If a collision is detected: the station stops transmitting immediately; it applies the back-off procedure; it waits for a suitable (randomized) time; and it then retransmits the frame.

Worked Example

Suppose Station A starts transmitting. Before A's signal has had time to physically reach Station C, C independently senses the channel as (still) idle and also begins transmitting. The two signals collide somewhere on the medium. C detects the collision as soon as it receives A's (garbled, overlapping) signal, and stops transmitting; likewise, A detects the collision once it receives C's signal, and it too stops transmitting. Both stations subsequently retransmit later, each using its own back-off procedure.

Comparison of CSMA Persistence Methods

Method	If Channel is Idle	If Channel is Busy	Collision Probability
1-Persistent	Transmit immediately	Continuously sense until idle	High
Nonpersistent	Transmit immediately	Wait a random time before sensing again	Low
p-Persistent	Transmit with probability **p**	Wait for next slot with probability **1 – p**	Moderate

Collision example: A and C both start transmitting before either can hear the other, causing a collision that both eventually detect.

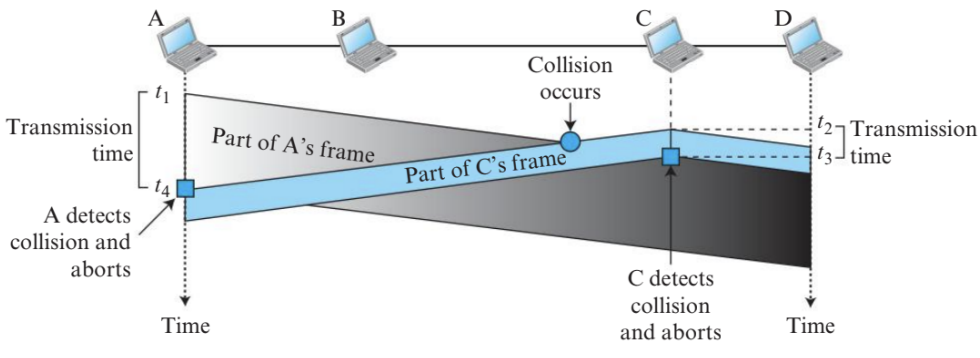
15.2 Minimum Frame Size Requirement

For CSMA/CD's collision-detection mechanism to work correctly at all, a minimum frame size is required. The core requirement is that the sender must still be in the process of transmitting the frame at the moment any collision signal returns to it — otherwise, the station would have already finished sending and moved on, with no opportunity left to detect and react to the collision. This gives the requirement:

$$T^f_r \geq 2T_p$$

where T^f_r is the frame transmission time and T_p is the maximum propagation time across the medium.

Figure 5.39 *Collision and abortion in CSMA/CD*



Minimum frame size in CSMA/CD: the frame transmission time must be at least twice the maximum propagation time (2Tp).

Why $2T_p$?

Consider the worst case, in which two stations, A and B, sit at the maximum possible distance from each other on the medium: A — T_p — B. A's signal takes T_p to physically reach B. If B then starts transmitting

(because, up to that point, B had genuinely sensed the channel as idle), a collision occurs at B's end; the resulting collision signal then has to travel all the way back from B to A, taking a further T_p. So the total round-trip time before A can possibly learn about the collision is:

$$\textbf{Total worst-case time} = T_p + T_p = 2T_p$$

Hence, the frame must be made long enough that its transmission time is at least this worst-case round-trip time — otherwise A could finish transmitting and disconnect from the medium before the collision notification even has a chance to reach it, defeating the entire purpose of collision detection. This $2T_p$ requirement is a key design constraint of traditional shared-medium (coaxial or hub-based) Ethernet, and directly explains why Standard Ethernet specifies a minimum frame size of 64 bytes, as discussed later in this module.

16. CSMA/CA (Collision Avoidance)

CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) is a variant of CSMA used mainly in wireless LANs. Rather than detecting collisions after they occur (as CSMA/CD does), its underlying philosophy is to actively avoid collisions before they can happen in the first place — this is necessary because, unlike a wired Ethernet transceiver, a typical wireless radio cannot transmit and listen (receive) on the same channel simultaneously, making true 'collision detection' impractical over the air. CSMA/CA and the IEEE 802.11 wireless LAN standard that uses it are covered in detail in the Wireless LANs section of this module (Section 20).

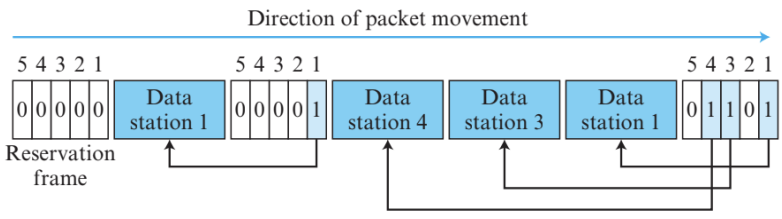
17. Controlled Access Protocols

In controlled access, stations actively coordinate with one another (or with a designated controller) to decide which single station is allowed to transmit at any given moment. A station is simply not permitted to transmit until it has been granted explicit permission. There are three principal controlled-access methods: Reservation, Polling, and Token Passing.

17.1 Reservation

In the reservation method, a station must formally reserve the channel before it is allowed to transmit any data on it.

Figure 5.42 *Reservation access method*



Reservation access method: a reservation frame with N minislots precedes the data frames in each interval.

- Time on the medium is divided into a sequence of intervals.

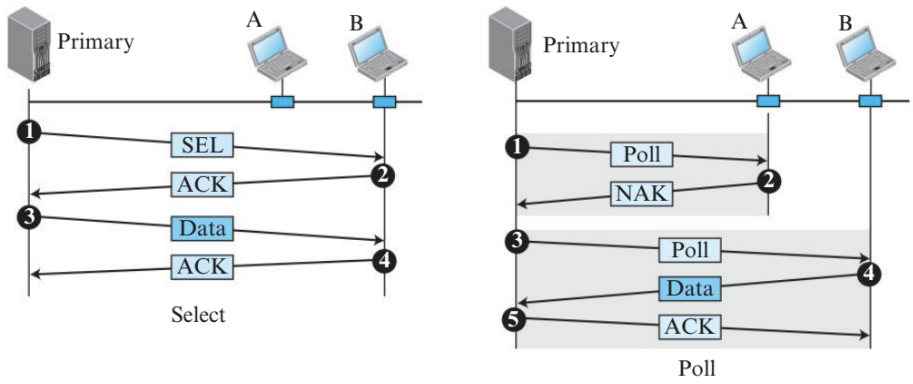
- Each interval begins with a short reservation frame, followed by the actual data frames for that interval.
- If there are N stations sharing the medium, the reservation frame is itself subdivided into N minislots, one dedicated minislot per station.
- A station that wishes to transmit during the upcoming interval simply marks (makes a reservation in) its own designated minislot.
- After the reservation frame has been sent, only those stations that made a reservation are permitted to transmit their data during that interval.

Example: if there are 5 stations in total, and only stations 1, 3, and 4 make reservations for a given interval, then only these three stations get to transmit data during that interval — stations 2 and 5 must wait for a future interval.

17.2 Polling

In polling, one device in the network is designated the primary station, while all the remaining devices act as secondary stations.

Figure 5.43 *Select and poll functions in polling-access method*



Polling: the primary station uses Select to send data to a secondary, and Poll to request data from a secondary.

- The primary station controls all access to the shared channel.
- Secondary stations are only permitted to transmit when the primary explicitly grants them permission.
- Since the primary is the sole arbiter of who transmits and when, collisions on the medium are inherently prevented.
- The main disadvantage of polling is its single point of failure: if the primary station itself fails, the entire system can stop functioning, since no secondary station can initiate communication on its own.

Polling relies on two distinct functions, Select and Poll, used depending on the direction of data flow:

a) Select

Select is used whenever the primary station itself has data that it wants to send to one particular secondary station.

- The primary prepares the data it wants to send.
- The primary sends a SEL (Select) frame that contains the address of the intended secondary recipient.
- That secondary station responds, indicating whether it is currently ready to receive.
- Once the primary receives this 'ready' response, it goes ahead and sends the actual data.

Note: Simple idea — Select is essentially the primary asking a secondary station, 'Are you ready to receive my data?'

b) Poll

Poll is used, conversely, whenever the primary station wants to receive data from one of the secondary stations.

- The primary asks each secondary station, in turn, whether it currently has any data that it wants to send.
- If a given secondary has no data to send, it responds with a NAK, and the primary simply moves on to poll the next secondary station in sequence.
- If a secondary does have data ready, it sends its data frame in response to being polled.
- The primary receives this data and, in turn, sends back an ACK.

17.3 Token Passing

Token passing is a controlled-access method in which the participating stations are conceptually arranged into a logical ring. A special, small control packet called a token continuously circulates around this ring from station to station. A station is only permitted to transmit data while it is actually holding the token — this guarantees that, at any given moment, at most one station can possibly be transmitting, so collisions are completely avoided by construction.

Working of Token Passing

- Every station has exactly one predecessor (the previous station in the ring) and exactly one successor (the next station in the ring).
- The token is passed sequentially from each station to its successor around the ring.
- If a station has data ready to send, it simply waits until it next receives the token.
- Once the token arrives, the station holds onto it and transmits its data.
- When the station has finished sending (or has no more data to send at that moment), it releases the token, forwarding it on to its successor.
- A station that currently has no data to send simply passes the token straight on to the next station without transmitting anything.
- Having transmitted once, a station must then wait to receive the token again — going all the way around the ring — before it is permitted to transmit another batch of data.

Token Management

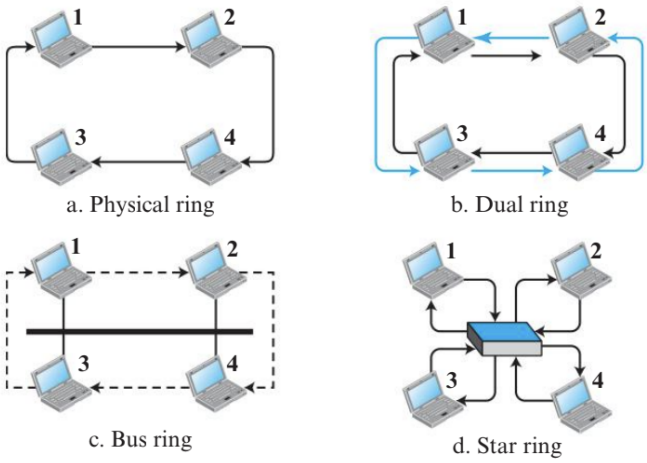
Proper functioning of a token-passing network requires active token management, in order to:

- Limit the maximum length of time any one station is allowed to hold the token, preventing monopolization.
- Detect situations where the token has been lost or corrupted/destroyed, and regenerate a fresh token in such cases.
- Manage relative priorities among different stations and different classes of data.
- Ensure that lower-priority stations release the token promptly when a higher-priority station needs access to it.

Logical Ring vs. Physical Topology

It is important to note that the stations participating in token passing do not need to be physically wired together in an actual ring shape — the 'ring' referred to here is purely logical (i.e., the order in which the token circulates). Several different physical topologies can be used to implement this same logical ring.

Figure 5.44 *Logical ring and physical topology in token-passing access method*



Four physical topologies that can implement a logical token-passing ring: physical ring, dual ring, bus ring, and star ring.

1. Physical Ring

- Stations are physically wired together in an actual ring layout.
- The token is passed directly, in sequence, from each station straight to the physically next station.
- Because the physical wiring order matches the logical order, the token itself does not even need to explicitly carry the address of the intended successor.
- Disadvantage: the failure of even a single link in the ring can potentially bring down the entire network, since the ring's continuity is broken.

2. Dual Ring

- Uses two separate concentric rings: a main ring, which is used under normal operation, and an auxiliary ring, which normally sits unused.
- If a link on the main ring fails, the two rings can be combined together (looped back at the point of failure) to form one single temporary ring, keeping the network operational.
- Once the failed link is repaired, the auxiliary ring reverts to its normal idle state.
- This resilience does come at a cost: each station needs two transmitters and two receivers instead of just one of each.
- Used in FDDI (Fiber Distributed Data Interface) and CDDI (Copper Distributed Data Interface).

3. Bus Ring (Token Bus)

- Stations are physically connected to a single, shared bus cable (physically resembling a bus topology, not a ring).
- Logically, however, the stations still form a ring, because each station is configured to know the address of its logical successor.
- When a station finishes its transmission, it passes the token onward by addressing a special token frame to the address of the next station in the logical sequence.
- Only the station whose address matches the one named in the token frame is permitted to access the medium next.
- Used in Token Bus LAN, an IEEE-standardized LAN technology.

4. Star Ring

- Physically, the network is wired as a star topology, with a central hub connecting every station.
- The logical ring is actually implemented by the internal wiring inside the hub itself, rather than by the external cabling to each station.
- If one link (spoke of the star) fails, the central hub is able to simply bypass the failed link internally, allowing every other station to continue operating normally.
- Stations can be added to or removed from the network considerably more easily than with a physical ring, since each station only has one cable running to the central hub.
- Used in IBM Token Ring LAN.

18. Link-Layer Addressing

At the Network Layer, devices are identified using IP addresses, which identify the ultimate source and final destination of a packet across the entire Internet. However, IP addresses alone are not sufficient to actually deliver a packet, because a single packet's journey typically crosses several different links and passes through several routers along the way.

For this reason, the Data Link Layer relies on a different kind of address, called the link-layer address — also referred to as the link address, the physical address, or (most commonly) the MAC address.

18.1 Why Do We Need Link-Layer Addresses?

- An IP address identifies the ultimate source and the final destination of a packet, for the purpose of routing across the whole internetwork.
- A link-layer address identifies only the next node on the current link — i.e., the device that should receive the frame at the very next hop.
- As a packet moves from one link to the next, passing through an intervening router, its IP source and destination addresses remain completely unchanged throughout the journey.
- However, the link-layer source and destination addresses change at every single link along the path, since each link has its own local set of directly-connected devices.

Example

Suppose:

Alice → Router 1 → Router 2 → Bob

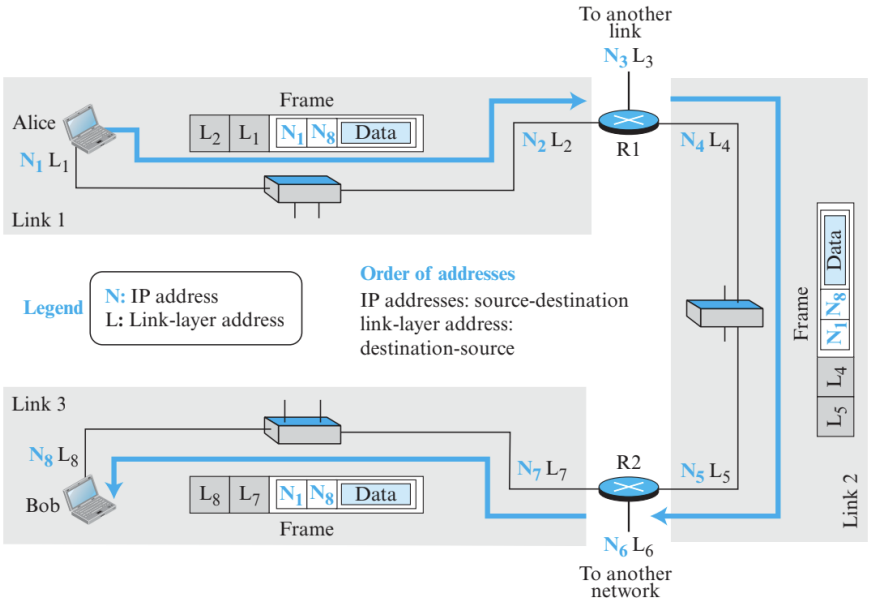
The same IP datagram travels through all three links:

Link	Link-Layer Destination	Link-Layer Source
Link 1	L2	L1
Link 2	L5	L4
Link 3	L8	L7

IP addresses stay the same end-to-end, while the link-layer (MAC) addresses change at every hop along the path.

For example, if a packet is travelling from node N1 to node N8 across several intermediate links and routers, the packet's IP source address (N1) and IP destination address (N8) remain fixed throughout, but the MAC source/destination address pairs are entirely different on each individual link along the way, since each link only needs to identify its own two immediately-connected endpoints.

Figure 5.45 *IP addresses and link-layer addresses in a small internet*



At every router, the old link-layer frame is stripped away, the datagram is extracted, and a brand-new frame with new link-layer addresses is created for the next link.

When a datagram passes down from the Network Layer to the Data Link Layer for transmission, it is encapsulated into a frame, with appropriate link-layer addresses attached. At every router the datagram passes through: the old incoming frame is discarded, the datagram itself is extracted unchanged, and a brand-new frame — carrying new, locally-appropriate source and destination link-layer addresses for the next link — is constructed before the datagram is forwarded onward.

IP Address vs. Link-Layer Address — Summary

IP Address	**Link-Layer Address**
Operates at the Network Layer	Operates at the Data Link Layer
Identifies the source and final destination	Identifies nodes on the current link only
Remains unchanged from source to destination	Changes at every hop
Used for routing across the internetwork	Used for delivery over one particular link
The same IP protocol works throughout the whole Internet	Depends on which data-link protocol is being used
Example: an IPv4 address	Example: a MAC address

18.2 Why Does a Router Have Multiple IP Addresses?

A router typically has multiple interfaces, with each interface connecting to a different link — and consequently, a router generally needs one distinct IP address per interface, since IP addressing is inherently tied to the specific link/subnet a device is directly attached to. For example, a router with 3 separate interfaces, connecting it to 3 different networks, will typically have 3 separate IP addresses, one per interface.

How Are the Relevant Addresses Determined?

- Source IP address: the sending host simply knows its own IP address, since it is configured on the host itself.
- Destination IP address: usually obtained through a DNS lookup, and then handed down to the Network Layer for use.
- Source link-layer address: the device already knows its own MAC address, since it is burned into (or configured on) its own network interface.
- Destination link-layer address: in IPv4 networks, this is determined dynamically using the Address Resolution Protocol (ARP), covered in detail next.

The size of a link-layer address depends entirely on which data-link protocol is in use for that particular link — different link technologies may define different address formats and different address lengths.

> *Note: In simple terms: the IP address tells a packet where it ultimately needs to go (across the whole Internet), while the link-layer/MAC address tells a given frame exactly which device to deliver it to on the current, local link.*

19. Address Resolution Protocol (ARP)

ARP (Address Resolution Protocol) is used to discover the link-layer (MAC) address that corresponds to a known IP address, for a device on the same local network.

19.1 Why Is ARP Needed?

When a host or router needs to send a packet onward to the next node, it typically already knows that next node's IP address (obtained, for example, via routing-table lookup), but it does not necessarily know that node's MAC address, which is what is actually needed to build the outgoing link-layer frame. ARP exists precisely to solve this problem:

`IP Address → ARP → MAC Address`

In effect, ARP supplies the missing MAC address to the Data Link Layer, so that the outgoing frame can actually be addressed and delivered correctly over the local link.

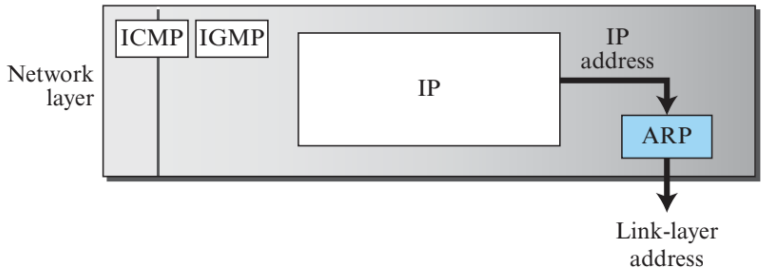
19.2 Working of ARP

Suppose host A wants to send data to host B, both on the same local network:

- A already knows B's IP address (say, N2), but does not know B's MAC address.
- A constructs an ARP Request, effectively asking, 'Who has IP address N2?'
- This ARP request is sent as a broadcast to every device on the local network — since A does not yet know which specific device is B, it must ask everyone.
- Every host and router on the local network receives this broadcast request.
- Only B recognises N2 as its own IP address; every other device silently ignores the request.
- B replies with an ARP Reply, containing both its own IP address and its own MAC address.

- Crucially, this reply is sent as a unicast, addressed directly and only back to A (since B, from the request, now knows exactly who asked and where to reply).
- A now has B's MAC address, and can proceed to construct and send the actual data frame directly to B.

Figure 5.46 *Position of ARP in TCP/IP protocol suite*



ARP working: A broadcasts an ARP request for B's IP address; only B responds, with a unicast ARP reply containing its MAC address.

Simple Worked Example

A: 'Who has IP 192.168.1.20?' — sent as a broadcast ARP Request.

B: 'I have 192.168.1.20. My MAC address is XX:XX:XX:XX:XX:XX.' — sent as a unicast ARP Reply, directly back to A.

A can now address and send the actual data frame directly to B's MAC address.

ARP Request vs ARP Reply

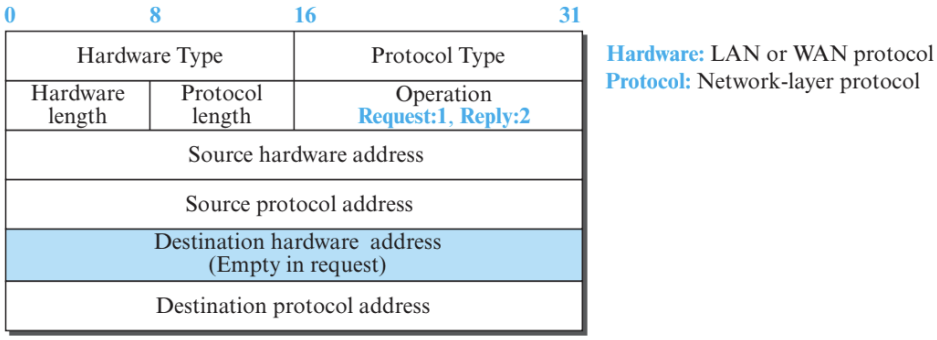
ARP Request	ARP Reply
Sent by the device that needs the MAC address	Sent by the device whose IP address was requested
Broadcast	**Unicast**
Contains the target IP address	Contains target's IP and MAC address
Asks: "Who has this IP?"	Answers: "I have this IP; this is my MAC."

A concrete ARP request/reply exchange, showing the broadcast request and the unicast reply.

19.3 ARP Packet Format

An ARP packet carries the following pieces of information about both the sender and the intended receiver:

Figure 5.48 *ARP packet*



Fields of an ARP packet: hardware type, protocol type, and the sender/target hardware and protocol addresses.

- Hardware Type — identifies the link-layer protocol in use; Ethernet is represented by the value 1.
- Protocol Type — identifies the Network-Layer protocol in use; IPv4 is represented by the value 0800_{16}.
- Source Hardware Address — the sender's own MAC address.
- Source Protocol Address — the sender's own IP address.
- Destination Hardware Address — the intended receiver's MAC address (left blank/unknown in the initial request, and filled in by the reply).
- Destination Protocol Address — the intended receiver's IP address (this is the one thing the sender already knows, and is trying to resolve to a MAC address).

An ARP packet is encapsulated directly inside a Data Link Layer frame — the frame itself carries a field (such as the Type field in an Ethernet frame) that indicates its payload is specifically an ARP packet, and not an ordinary IP datagram.

> *Note: ARP is often described as an 'auxiliary' protocol associated with the Network Layer, but its true purpose is to bridge Network-Layer addressing (IP) together with Data-Link-Layer addressing (MAC). In short: ARP = IP address → MAC address; an ARP Request is always a broadcast; an ARP Reply is always a unicast.*

20. Wired LANs: The Ethernet Protocol

A Local Area Network (LAN) is a network designed to cover a limited geographical area, such as a single building, office, or a college campus; most LANs are, in turn, connected onward to a WAN or to the wider Internet. The TCP/IP protocol suite deliberately does not mandate any one particular protocol for the Data Link Layer or the Physical Layer — it can work with whichever suitable protocol provides the required services to the Network Layer above it. LAN technologies can be either wired or wireless; this section focuses mainly on wired LANs.

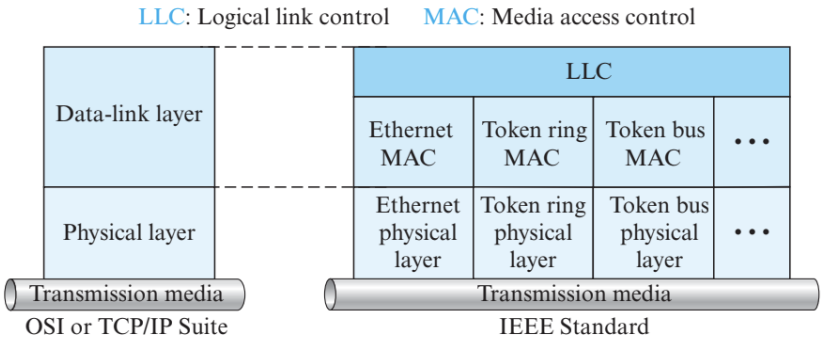
20.1 The Rise of Ethernet

During the 1980s and 1990s, several competing LAN technologies were in common use, each with its own medium-access method:

LAN Technology	Medium-Access Method
Ethernet	CSMA/CD
Token Ring	Token passing
Token Bus	Token passing
FDDI	Token passing
ATM LAN	ATM technology

Most of these alternative technologies have since largely disappeared from common use, while Ethernet became the dominant wired LAN technology — chiefly because Ethernet was able to evolve smoothly to support ever-increasing data rates without forcing organisations to discard and replace their entire existing infrastructure. Ethernet itself was originally developed in the 1970s by Robert Metcalfe and David Boggs.

Figure 5.54 *IEEE standard for LANs*



Evolution of Ethernet through successive generations, each providing a higher data rate.

Generation	Data Rate
Standard Ethernet	10 Mbps
Fast Ethernet	100 Mbps
Gigabit Ethernet	1 Gbps
10-Gigabit Ethernet	10 Gbps

20.2 IEEE Project 802

In 1985, the IEEE Computer Society launched Project 802, aimed at developing standards that would let networking equipment from different manufacturers interoperate correctly with one another. Project 802 does not replace either the OSI or the TCP/IP reference models; instead, it specifically fleshes out the details of the Physical Layer and the Data Link Layer for various LAN technologies.

Under IEEE 802, the Data Link Layer is further split into two sublayers:

1. Logical Link Control (LLC)

LLC handles flow control, error control, and part of the framing function. It provides a common link-layer control interface shared across all IEEE LAN technologies, which effectively hides the specifics of the underlying MAC layer from the layers above — allowing different LAN technologies underneath to interoperate more easily with a common upper interface.

Note: LLC = the common control layer shared across different kinds of LANs.

2. Media Access Control (MAC)

MAC controls exactly how stations gain access to the shared transmission medium, and it defines the specific medium-access method used by each particular LAN technology — for example, Ethernet uses CSMA/CD, while Token Ring and Token Bus both use token passing. The MAC sublayer also performs part of the framing function.

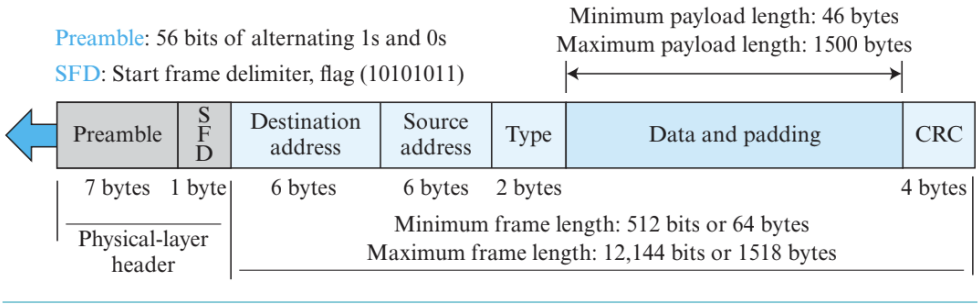
Note: MAC = the sublayer that controls access to the shared medium.

20.3 Standard Ethernet

Standard Ethernet is the original Ethernet technology, providing a data rate of 10 Mbps. Even though later Ethernet generations offer much higher data rates, many of the basic structural features first defined in Standard Ethernet — notably the frame format — remain common across all subsequent Ethernet generations.

Standard Ethernet Frame Fields

Figure 5.55 *Ethernet frame*



The Standard Ethernet frame format, showing the Preamble, SFD, Destination/Source Address, Type, Data, and CRC fields.

- Preamble (7 bytes / 56 bits) — an alternating pattern of 0s and 1s that alerts the receiving system that a frame is about to arrive and helps it synchronise its clock with the sender's. The preamble is technically added at the Physical Layer, and is not formally considered part of the Ethernet frame proper.
- Start Frame Delimiter, SFD (1 byte) — the fixed bit pattern 10101011, which marks the actual start of the Ethernet frame and gives the receiver one final opportunity to complete synchronisation. Like the preamble, the SFD is added at the Physical Layer.
- Destination Address, DA (6 bytes / 48 bits) — the MAC address of the intended destination station. The receiver accepts an incoming frame only if this address matches its own MAC address, a multicast group address it belongs to, or the broadcast address.

- Source Address, SA (6 bytes / 48 bits) — the MAC address of the sending station.
- Type (2 bytes) — identifies which upper-layer protocol's packet is being carried inside this Ethernet frame (for example, IP, ARP, or OSPF); this is what allows the receiver to multiplex/demultiplex incoming frames correctly to the right upper-layer protocol.
- Data (46 to 1500 bytes) — the actual payload handed down from the upper-layer protocol. If the upper-layer data is shorter than 46 bytes, extra padding bits are added to reach this minimum; if the data exceeds 1500 bytes, it must be split and carried across more than one Ethernet frame.
- CRC (4 bytes) — a CRC-32 value used for error detection, computed over the Destination Address, Source Address, Type, and Data fields together. The receiver recomputes this CRC on the received frame; if the check fails, the frame is deemed corrupted and is simply discarded.

Standard Ethernet — Important Characteristics

1. Connectionless and Unreliable Service

Ethernet provides a connectionless service — every frame is handled entirely independently of every other frame, with no formal connection-establishment or connection-termination phase involved at all. The sender is free to transmit a frame whenever it has data ready, without regard to whether the intended receiver happens to be ready for it at that instant. If a receiver is unable to keep up with incoming frames, some of them may simply be dropped, and Ethernet itself does not inform the sender of any such loss.

Ethernet is likewise unreliable, because it makes no attempt to retransmit lost or corrupted frames on its own. If a frame is corrupted in transit, the receiver's CRC-32 check detects the error, and the frame is silently discarded — recovering from that loss is left entirely to whichever higher-layer protocol is in use. For instance, UDP itself provides no retransmission, so a UDP-based application may need to cope with the missing data on its own; TCP, on the other hand, will notice the loss (because the expected acknowledgment never arrives) and will retransmit the corresponding segment automatically.

2. Frame Length

Ethernet specifies both a minimum and a maximum permissible frame length.

The minimum Ethernet frame length is 64 bytes (512 bits). This particular minimum is required specifically for CSMA/CD's collision-detection mechanism (the $2T_p$ requirement covered earlier in Section 15.2) to function correctly. The Ethernet header and trailer together occupy 18 bytes in total: 6 bytes for the Destination Address, 6 bytes for the Source Address, 2 bytes for the Type field, and 4 bytes for the CRC. This means the minimum size of the Data field alone works out to:

$$64 - 18 = 46 \text{ bytes}$$

If the actual upper-layer data supplied is shorter than 46 bytes, padding bits are automatically added to bring the frame up to the required minimum size.

The maximum Ethernet frame length is 1518 bytes, excluding the Preamble and SFD (which, as noted, are not formally part of the frame). Subtracting the same 18 bytes of header and trailer overhead:

$$1518 - 18 = 1500 \text{ bytes}$$

Therefore, the maximum payload (Data field) size is 1500 bytes. This maximum was originally introduced for two main reasons: first, memory was expensive at the time Ethernet was designed, so limiting the frame

size helped keep the required buffer sizes manageable; and second, it prevents any single station from monopolizing the shared medium for an excessively long, uninterrupted stretch of time, which would be unfair to other stations waiting to transmit.

3. Ethernet Addressing

Every Ethernet station — a computer, workstation, printer, or any other networked device — has a Network Interface Card (NIC), and it is this NIC that provides the station with its link-layer (MAC) address.

An Ethernet MAC address is 6 bytes (48 bits) long, and is conventionally written in hexadecimal notation, with a colon separating each successive byte, e.g., XX:XX:XX:XX:XX:XX. This address uniquely identifies one particular network interface on the Ethernet network.

4. Transmission of Address Bits

The order in which a MAC address is actually transmitted over the wire differs from the order in which it is conventionally written down. The address is transmitted byte by byte, from left to right (i.e., the leftmost written byte goes out first); however, within each individual byte, it is the least significant bit (LSB) that is transmitted first, and the most significant bit (MSB) that is transmitted last. Because of this convention, the very first bit received by the far end immediately tells the receiver whether the address in question represents a unicast or a multicast transmission.

5. Efficiency of Standard Ethernet

The efficiency of Ethernet is defined as the ratio of the time a station actually spends transmitting useful data, to the total time for which the shared medium is effectively occupied because of that station's transmission (including contention/idle overhead). This efficiency depends heavily on a parameter conventionally called a, which captures the relationship between propagation delay and transmission delay:

```
a = Propagation Delay / Transmission Delay
```

Here, transmission delay is the time required to transmit one entire frame, while propagation delay is the time required for the signal to travel across the length of the medium. The key relationship to remember is: a smaller value of a leads to higher Ethernet efficiency. Efficiency therefore improves when the medium is physically shorter (reducing propagation delay), or when the frame size is larger (increasing transmission delay). In the idealised limiting case where a = 0, efficiency reaches its theoretical maximum of 1 (100%).

20.4 Fast Ethernet (100 Mbps)

Fast Ethernet is the second generation of Ethernet, introduced during the 1990s to provide a data rate ten times higher than the original 10 Mbps Standard Ethernet, while still remaining compatible with it wherever possible.

Main Features

The designers deliberately kept the MAC sublayer essentially unchanged from Standard Ethernet, retaining the same Ethernet frame format and the same minimum/maximum frame sizes, in order to preserve compatibility. However, achieving the tenfold increase in transmission rate necessitated changes to the access method's timing parameters and to the Physical Layer implementation.

Access Method

CSMA/CD's correct operation depends critically on the relationship between transmission rate, minimum frame size, and the maximum network length (via the $2T_p$ rule). Since Fast Ethernet transmits ten times faster than Standard Ethernet, the same 512-bit minimum frame is now transmitted ten times more quickly — meaning any resulting collision must also be detected within a correspondingly shorter time window. To preserve the same minimum frame size (and thus frame-format compatibility) while still satisfying the $2T_p$ requirement at the new, higher speed, the maximum permissible network length had to be reduced by roughly the same factor of 10:

Ethernet Generation	**Maximum Network Length**
Standard Ethernet	2500 m
Fast Ethernet	250 m

Table 5.7 *Summary of Fast Ethernet implementations*

Implementation	*Medium*	*Medium Length*	*Wires*	*Encoding*
100Base-TX	STP	100 m	2	4B5B + MLT-3
100Base-FX	Fiber	185 m	2	4B5B + NRZ-I
100Base-T4	UTP	100 m	4	Two 8B/6T

Two approaches used by Fast Ethernet: a shared hub-based star topology, versus a switched full-duplex configuration.

Two Approaches Used by Fast Ethernet

- 1. Shared Medium with Hub — this approach abandons the older bus topology in favour of a star topology built around a passive hub, with the maximum network length reduced to 250 m (from 2500 m). This approach retains CSMA/CD and remains directly compatible with Standard Ethernet's access method.
- 2. Switched Full-Duplex Ethernet — this second approach instead uses a link-layer switch equipped with per-port buffers, giving each host its own dedicated, full-duplex connection to the switch. Because every host has a private link to the switch (rather than sharing a single medium with everyone else), there is no contention between hosts for the medium at all.

In this switched approach, the switch: receives an incoming frame from the source; temporarily stores it in a buffer/queue; checks the frame's destination MAC address; and then forwards the frame out through only the appropriate port. Because each connection to the switch is full-duplex, a station is able to send and receive simultaneously; consequently, there is no contention on any individual link, and CSMA/CD is not required at all in this mode.

Autonegotiation

Autonegotiation is a feature introduced alongside Fast Ethernet that allows two directly connected devices to automatically negotiate, between themselves, the data rate and operating mode (half-duplex vs. full-duplex) they will use to communicate. For example, a device that only supports 10 Mbps can still successfully communicate with a 100 Mbps-capable device, provided the latter also supports falling back to operate at 10 Mbps.

20.5 Gigabit Ethernet (1 Gbps)

Continuing demand for ever-higher transmission rates led to the development of Gigabit Ethernet, providing a data rate of 1000 Mbps (1 Gbps); its IEEE standard designation is 802.3z.

Table 5.8 *Summary of Gigabit Ethernet implementations*

Implementation	*Medium*	*Medium Length*	*Wires*	*Encoding*
1000Base-SX	Fiber S-W	550 m	2	8B/10B + NRZ
1000Base-LX	Fiber L-W	5000 m	2	8B/10B + NRZ
1000Base-CX	STP	25 m	2	8B/10B + NRZ
1000Base-T4	UTP	100 m	4	4D-PAM5

Gigabit Ethernet retains the same 48-bit addressing and frame format as earlier Ethernet generations, while raising the data rate to 1 Gbps.

The main design goals were: to raise the data rate to 1 Gbps; to keep the same 48-bit MAC address length; to keep the same Ethernet frame format; and to keep the same minimum and maximum frame sizes as before.

MAC Sublayer

At 1 Gbps, it proved genuinely difficult to keep the original shared-medium CSMA/CD access method unchanged while still meeting the $2T_p$ timing constraint over any reasonable network length. Gigabit Ethernet does technically support both half-duplex and full-duplex modes, but in practice, almost all real-world implementations use full-duplex mode exclusively.

In full-duplex Gigabit Ethernet, a central switch connects together all the computers (or other switches) in the network; this switch maintains input buffers at each of its ports, to temporarily hold incoming frames. The switch examines each frame's destination MAC address and forwards it out through the correspondingly appropriate port only. Because every connection to the switch is dedicated and full-duplex, there is no collision at all, CSMA/CD is not required, and the maximum usable cable length ends up being determined mainly by ordinary signal attenuation, rather than by any collision-detection timing constraint.

20.6 10-Gigabit Ethernet

10-Gigabit Ethernet was developed both to provide a transmission rate of 10 Gbps and to extend the reach of the Ethernet technology well beyond traditional LAN environments alone. Its IEEE standard designation is 802.3ae.

Table 5.9 *Summary of 10-Gigabit Ethernet implementations*

Implementation	*Medium*	*Medium Length*	*Number of wires*	*Encoding*
10GBase-SR	Fiber 850 nm	300 m	2	64B66B
10GBase-LR	Fiber 1310 nm	10 Km	2	64B66B
10GBase-EW	Fiber 1350 nm	40 Km	2	SONET
10GBase-X4	Fiber 1310 nm	300 m to 10 Km	2	8B10B

10-Gigabit Ethernet operates only in full-duplex mode, so there is no contention and CSMA/CD is not used at all.

Main Goals

- Provide a data rate of 10 Gbps.
- Retain the same Ethernet frame format and the same frame size limits as earlier generations.
- Support interconnection not just of LANs, but also of MANs and WANs.
- Use fiber-optic transmission technology throughout.

Two distinct Physical Layer variants are defined: LAN PHY, designed for supporting existing LAN environments; and WAN PHY, designed for supporting WAN-style connections that make use of SONET OC-192 framing.

Access Method

10-Gigabit Ethernet operates exclusively in full-duplex mode. As a direct consequence: there is no shared medium at all, hence no contention among stations, hence no possibility of collisions, and hence CSMA/CD is simply not used in 10-Gigabit Ethernet.

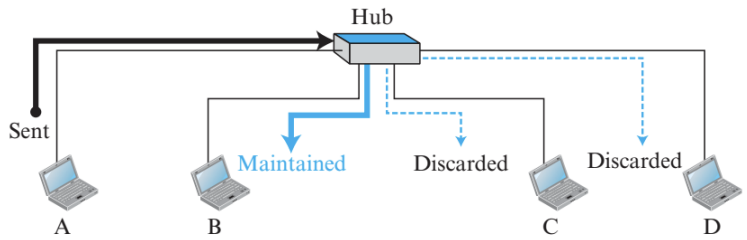
21. Connecting Devices

Hosts and networks rarely operate in complete isolation; connecting devices are used both to link individual hosts together to form a single network, and to link entirely separate networks together to form a larger internetwork. Different connecting devices operate at different layers of the Internet model, and the three most important such devices — repeaters/hubs, link-layer switches, and routers — are discussed below. A repeater or hub operates only at the Physical Layer; a link-layer switch operates at both the Physical and Data Link Layers; and a router (also called a three-layer switch) operates at the Physical, Data Link, and Network Layers.

21.1 Repeater or Hub

A repeater is a device that operates purely at the Physical Layer. Electrical/optical signals carrying data can only travel a limited physical distance before they weaken or become corrupted by noise. A repeater receives such a weakened signal, regenerates and correctly retimes the original bit pattern, and then forwards the freshly-regenerated signal onward. In older Ethernet networks built around a bus topology, repeaters were used to connect separate network segments together, overcoming cable-length limitations.

Figure 5.84 *Repeater or hub*



A hub simply regenerates an incoming signal and rebroadcasts it out through every other port; it has no filtering capability.

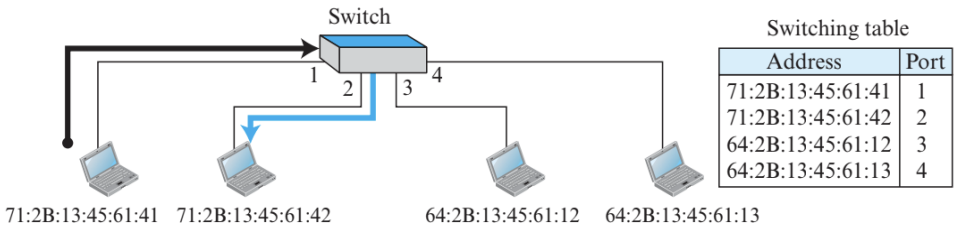
In modern Ethernet networks, which typically use a star topology instead of a bus, a repeater usually takes the form of a multiport device called a hub. When a frame arrives at a hub through one of its ports, the hub regenerates the signal and forwards it out through every other port simultaneously. As a result, every connected device receives a copy of the frame, but only the actual intended destination accepts and processes it — every other device simply discards its copy. A hub has no filtering capability whatsoever, because it never examines MAC addresses at all; it purely regenerates and broadcasts the incoming signal

out through all outgoing ports. Since a hub operates only at the Physical Layer, it has no concept of, and makes no use of, MAC addresses.

21.2 Link-Layer Switch

A link-layer switch, also called a two-layer switch, operates at both the Physical Layer and the Data Link Layer. At the Physical Layer, like a repeater, it regenerates the received signal; at the Data Link Layer, it goes further and examines the source and destination MAC addresses actually present in the frame. The chief advantage a switch has over a plain hub is its filtering capability: a switch maintains an internal table mapping MAC addresses to the specific ports they are reachable through. When a frame arrives, the switch checks the frame's destination MAC address against its table, and forwards the frame out through only the one appropriate port, rather than blindly broadcasting it out through every port as a hub would.

Figure 5.85 *Link-Layer Switch*



A link-layer switch consults its switching table to forward each frame out only through the port where the destination is known to be located.

For example, if a frame destined for a particular station arrives at port 1 of the switch, and the switch's table shows that the destination station is actually reachable via port 2, the switch forwards that frame out through port 2 only — every other port stays silent. In this way, a switch substantially reduces unnecessary traffic on the network compared with a hub. Importantly, a link-layer switch never alters the MAC addresses carried inside a frame.

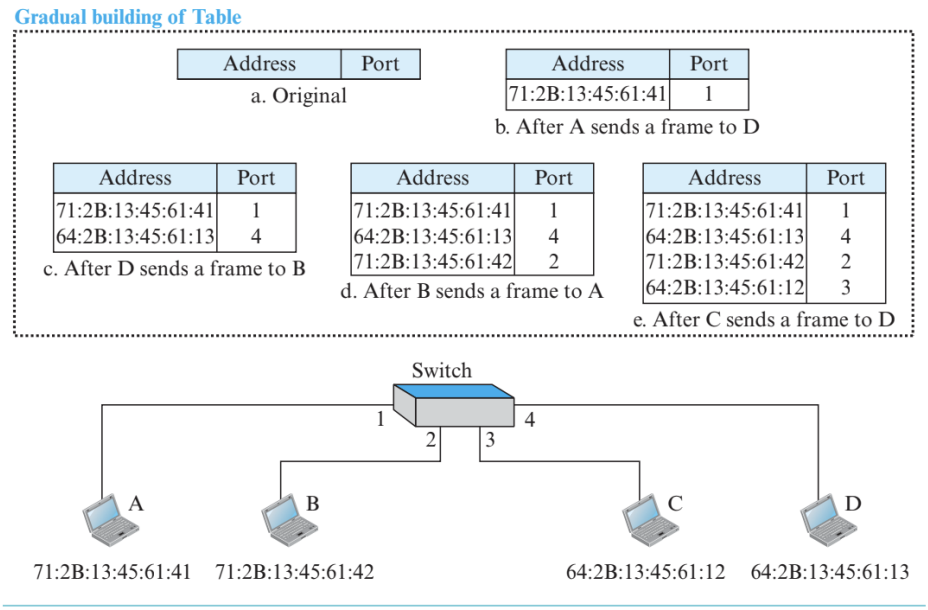
Transparent Switch

A transparent switch is one whose very presence is completely invisible to the stations connected to it — those stations do not need to know a switch even exists on the network, and adding or removing such a switch requires no reconfiguration of the stations themselves. A transparent switch carries out three principal functions: forwarding (correctly delivering frames to their intended destination); learning (automatically building up its forwarding table simply by observing frames as they move through the network); and loop prevention (ensuring frames cannot circulate endlessly around the network whenever multiple redundant paths happen to exist).

Learning in a Switch

Modern switches build their MAC address table dynamically, rather than relying on a manually configured static table. The switch learns which device is connected to which port purely by examining the source MAC address of every incoming frame it sees.

Figure 5.86 *Learning switch*



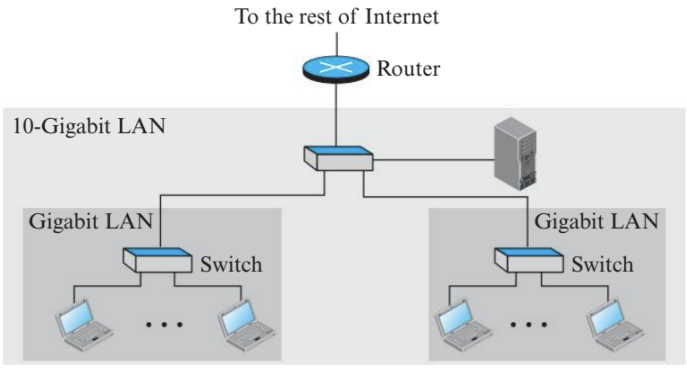
As stations transmit, the switch progressively learns their locations from source addresses, gradually building a complete forwarding table.

The destination MAC address of a frame is used to decide where that frame should be forwarded to, while the source MAC address of the very same frame is simultaneously used to learn/update the switch's own table. For example, when station A sends a frame addressed to station D, and the switch does not yet know the location of either A or D, the switch simply floods the frame out through all of its other ports (rather than filtering). At the same moment, by examining the frame's source address, the switch learns that station A is connected via port 1, and it records this fact in its table. Later, when station D happens to send a frame to station B, the switch similarly learns which port D is connected through. Over time, as more and more stations transmit frames, the switch progressively learns the location of every device and builds up a complete forwarding table. Whenever the switch does not (yet) know a given destination's MAC address, it simply floods that particular frame out to every other port, exactly as a hub would.

21.3 Router

A router, also referred to as a three-layer device, operates at the Physical, Data Link, and Network Layers all together. At the Physical Layer, it regenerates signals; at the Data Link Layer, it examines MAC addresses; and at the Network Layer, it additionally examines IP addresses. The primary purpose of a router is to connect together separate, distinct networks, and to allow packets to travel from one network onward to another — making it a critically important internetworking device.

Figure 5.87 *Routing example*



A router connects separate networks together, examining both MAC and IP addresses, and constructing a new frame for each outgoing link.

There are three major differences between a router, and a hub or switch: first, a router has both a physical (MAC) address and a logical (IP) address associated with each of its interfaces (whereas a hub has neither, and a switch has only MAC addresses in its table, not IP addresses of its own). Second, a router only processes an incoming packet once the packet's link-layer destination address specifically matches the MAC address of the receiving interface. Third — and most importantly — when a router forwards a packet on to another network, it changes the link-layer (MAC) source and destination addresses, because MAC addresses only ever have local significance on one particular link; IP addresses, by contrast, are what actually enable communication across entirely different networks.

For example, suppose an organisation has two separate Ethernet LANs located in two different buildings. Switches can be used to interconnect the computers within each individual building. A high-speed Ethernet connection might then link the two buildings' LANs together, while a router separately connects the organisation's overall network onward to the Internet. Whenever a packet moves from one network to another, the router strips off the old data-link frame entirely and constructs a brand-new one, carrying the appropriate new source and destination MAC addresses needed for the next network — this rebuilding step is necessary precisely because MAC addresses only have local significance, whereas IP addresses are what remain valid for communication all the way across different networks.

22. Wireless LANs

A Wireless LAN (WLAN) is a network that connects devices together without the use of any physical cables. Wireless communication is among the fastest-growing networking technologies, driven by the increasing need to connect devices without the inconvenience of wires; wireless LANs are now commonplace in college campuses, offices, airports, hotels, and many other public spaces. Unlike a wired LAN, a wireless LAN uses the air itself as its communication medium.

22.1 Architectural Comparison of Wired and Wireless LANs

Medium

The most fundamental difference between wired and wireless LANs lies in the communication medium itself. In a wired LAN, hosts communicate over physical cables; in modern switched Ethernet, communication between any two hosts is generally point-to-point and full-duplex, meaning data can flow in both directions simultaneously along a dedicated link. In a wireless LAN, by contrast, the air serves as the medium, and wireless signals are inherently broadcast in nature — any device within range can potentially hear them. Since multiple wireless devices typically share the very same medium, wireless LANs necessarily require a multiple-access method, so that MAC protocols become essential for controlling how devices access this shared wireless medium.

Hosts and Mobility

In a wired LAN, a host is physically tied to the network via a cable and therefore has a fixed connection point; the host's link-layer (MAC) address is associated with its Network Interface Card (NIC), and if the host physically relocates to another network, its MAC address typically stays the same even though its network-layer (IP) address may well change. In a wireless LAN, a host requires no physical cable connection at all, and can freely move about within the wireless coverage area while remaining continuously connected — making mobility considerably easier to support in wireless LANs than in wired ones.

Isolated LANs

An isolated wired LAN is usually built from several hosts all connected through a common link-layer switch. An isolated wireless LAN, by contrast, can instead be formed as an ad hoc network, in which wireless hosts communicate directly with one another without requiring any switch at all — devices within an ad hoc network can communicate freely with any nearby device. Consequently, the traditional notion of a link-layer switch (as used in wired LANs) does not really apply in the same way to an isolated wireless LAN.

Connection to Other Networks

A wired LAN typically connects to another network, or to the Internet, via a router. A wireless LAN can likewise connect to other networks in several different ways — for instance, via a wired infrastructure network, via a wireless infrastructure network, or via another wireless LAN entirely. A very common real-world example is a wireless LAN in a college or an office, where wireless devices communicate through a wireless access point, which in turn connects them onward to the wired backbone network and, ultimately, to the Internet.

22.2 Infrastructure Network

When a wireless LAN is connected to a wired network, such as the Internet, through an Access Point (AP), the resulting arrangement is called an infrastructure network. The AP serves as the bridge between the wireless and wired environments: communication between a wireless host and the AP uses wireless signalling, while communication between the AP and the wired network beyond it uses ordinary cables.

Moving from Wired to Wireless

Both wired and wireless LANs primarily operate at the lowest two layers of the TCP/IP model. Converting a wired LAN into a wireless one, at a conceptual level, involves replacing the wired NIC with a wireless NIC, and replacing the switch with an access point. The MAC address naturally changes, since the NIC hardware itself has changed, but the IP address of the host remains the same throughout.

22.3 Characteristics of Wireless LANs

- Attenuation: wireless signals grow progressively weaker as they travel outward, since they spread out in multiple directions from the transmitter, rather than being confined to a single cable.
- Interference: signals coming from other, unrelated devices operating on the same frequency band can interfere with the desired signal, corrupting it.
- Multipath Propagation: a signal may reflect off walls, furniture, and other obstacles, and consequently arrive at the receiver via several different paths, each with a slightly different delay — making the overall received signal harder to correctly recognise and decode.
- Errors: because of the combined effects above, wireless networks tend to experience noticeably more transmission errors than wired networks; consequently, error detection, error correction, and retransmission all become correspondingly more important. A high Signal-to-Noise Ratio (SNR) indicates a strong, clean signal, whereas a low SNR indicates a weaker, noisier signal that is more prone to errors.

22.4 Access Control in Wireless LAN

Since wireless devices all share the same medium (the air), some access-control mechanism is required to decide, at any given moment, which device is permitted to transmit. Traditional wired Ethernet relies on CSMA/CD for this purpose, but CSMA/CD is not well suited to wireless LANs, essentially for three reasons: wireless devices generally cannot transmit and receive at the same time (making true, over-the-air collision detection impractical); the hidden-station problem (described below) can lead to undetected collisions; and signal fading can prevent a device from reliably detecting an ongoing collision even when one is occurring. For these reasons, wireless LANs instead use CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance), which is specifically designed to reduce the likelihood of collisions in the first place, rather than merely detecting them after the fact.

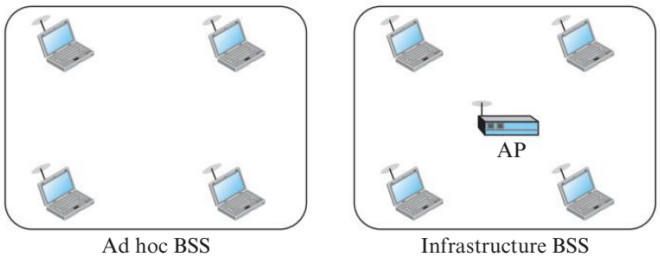
IEEE 802.11

IEEE 802.11 is the standard developed by the IEEE specifically for Wireless LANs (WLANs). It primarily defines the Physical Layer and Data Link Layer behaviour for such networks. 'Wi-Fi' itself refers to a WLAN technology that has been certified, for interoperability, by the Wi-Fi Alliance, based on the underlying IEEE 802.11 standard.

Architecture: BSS and ESS

IEEE 802.11 defines two principal types of wireless LAN architecture: the Basic Service Set (BSS) and the Extended Service Set (ESS).

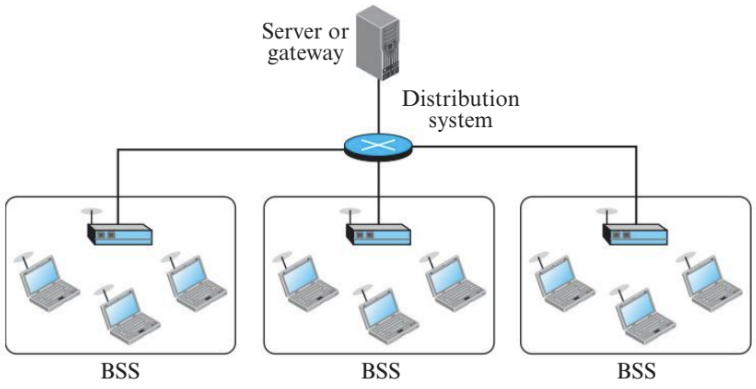
Figure 6.4 *Basic service sets (BSSs)*



Basic Service Set (BSS): the fundamental building block of a wireless LAN, optionally including an Access Point.

- Basic Service Set (BSS): the basic building block of a wireless LAN, consisting of a group of wireless stations, and optionally an Access Point (AP). A BSS that has no AP is called an ad hoc network, in which devices communicate directly with one another without any central coordinator. A BSS that does have an AP is called an infrastructure BSS, in which the AP connects the wireless devices onward to the broader network.
- Extended Service Set (ESS): consists of two or more BSSs, with each one having its own AP. These multiple APs are interconnected via a distribution system, which may itself be either wired or wireless. This arrangement allows devices located in different BSSs to communicate with one another, and provides an overall wireless network that is considerably larger than a single BSS. The distribution system itself might, for example, simply be an ordinary Ethernet network.

Figure 6.5 *Extended service set (ESS)*

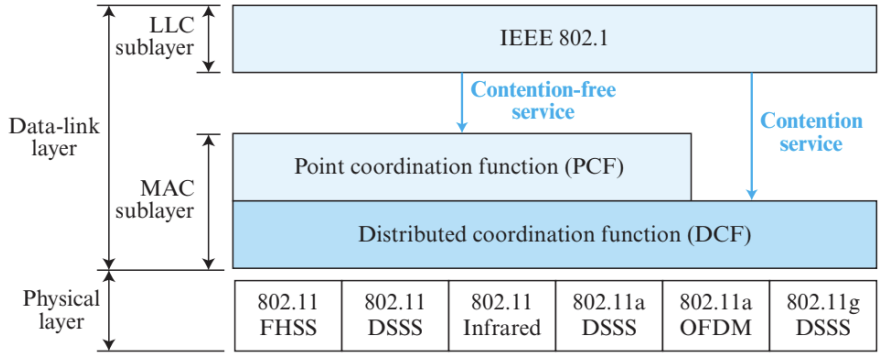


Extended Service Set (ESS): two or more BSSs, each with its own Access Point, interconnected via a distribution system.

Types of Mobility

IEEE 802.11 defines three distinct types of mobility: no-transition mobility, in which a device is either fully stationary, or only ever moves within the boundaries of a single BSS; BSS-transition mobility, in which a device may move from one BSS to another BSS, but both within the same overall ESS; and ESS-transition mobility, in which a device moves from one ESS entirely to another ESS — although in this last case, continuous, uninterrupted communication throughout the move is not actually guaranteed.

Figure 6.6 *MAC layers in IEEE 802.11 standard*



The three types of mobility defined by IEEE 802.11: no-transition, BSS-transition, and ESS-transition.

IEEE 802.11 MAC Sublayers

IEEE 802.11 defines two MAC sublayers: the Distributed Coordination Function (DCF), and the Point Coordination Function (PCF). DCF makes use of CSMA/CA and is the method commonly used for ordinary, everyday wireless communication. PCF, on the other hand, is an optional method mainly intended for time-sensitive communication that requires more predictable access.

DCF and CSMA/CA

DCF relies on CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) specifically to reduce the likelihood of collisions. It makes use of three main techniques working together: the Interframe Space (IFS), the Contention Window, and Acknowledgment.

Before transmitting, a station first checks whether the channel is idle. If it is idle, the station waits for a defined Interframe Space (IFS) duration, and then additionally selects a random waiting time drawn from within the contention window. Only after this combined waiting period does the station actually transmit. The receiver, upon successfully receiving the frame, sends back an ACK to confirm successful reception. If the sender fails to receive this ACK, it assumes the transmission failed (most likely due to a collision), and retransmits the frame after appropriately applying a back-off procedure.

RTS and CTS

CSMA/CA can additionally make use of RTS (Request to Send) and CTS (Clear to Send) control frames, specifically to further reduce the likelihood of collisions. The sender first transmits an RTS frame; the intended receiver responds with a CTS frame; only after receiving this CTS does the sender go ahead and transmit its actual data, after which the receiver sends back an ACK to confirm successful reception. The RTS/CTS mechanism is especially useful for solving the hidden-station problem, described next.

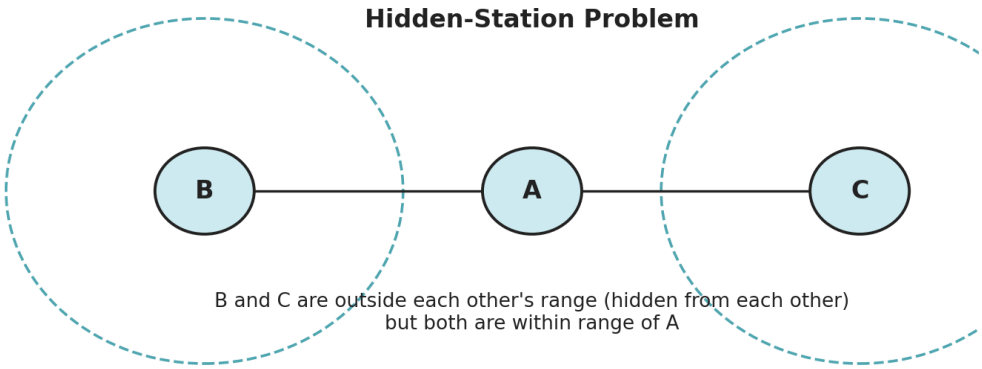
Network Allocation Vector (NAV)

The Network Allocation Vector (NAV) is a virtual timer used purely for collision avoidance purposes (it does not correspond to any actual, physical carrier-sensing activity). When a station transmits an RTS frame, that frame indicates precisely how long the station intends to occupy the channel for. Every other station that overhears this RTS (or the corresponding CTS) sets its own internal NAV timer accordingly, and simply waits out that indicated period before even attempting to transmit itself. In this way, NAV

effectively lets every other station know exactly when the channel is expected to become available again, without them needing to keep actively sensing the channel throughout that whole period.

Hidden-Station Problem

The hidden-station problem arises when two stations cannot directly hear (sense) each other's transmissions, and yet both of them are able to communicate with one and the same shared destination station.



The hidden-station problem: B and C cannot hear each other, but both can reach A.

For example, suppose stations B and C both lie outside each other's respective transmission range, but both are within range of a common station A. If B is currently sending data to A, station C may independently sense the channel as free (since it genuinely cannot hear B's ongoing transmission at all) and may therefore begin transmitting at the same time — causing a collision at A, even though B and C themselves never detect any conflict from their own individual vantage points. RTS/CTS specifically helps solve this problem: the CTS frame sent by A (in response to B's RTS) can, in fact, be heard directly by C — even though C could not hear B's original RTS — and this CTS informs C that the channel is about to be, or already is, in use, prompting C to defer its own transmission accordingly.

Point Coordination Function (PCF)

PCF is an optional access method that is only available within infrastructure networks (i.e., networks that include an AP) — it has no equivalent in a pure ad hoc network. PCF provides centralized, contention-free channel access, achieved through polling: the AP polls each station in turn, one by one, and grants it permission to transmit when polled. PCF uses a shorter interframe space called PIFS (PCF Interframe Space), which is deliberately made shorter than the DIFS used by DCF — this gives PCF-based traffic a higher priority than ordinary DCF-based traffic whenever both are in use on the same network. A special beacon frame marks the start of each repetition interval, and helps coordinate the alternation between contention-free (PCF) periods and contention-based (DCF) periods on the network.

22.5 IEEE 802.11 Physical Layer Variants

IEEE 802.11 has evolved through several distinct Physical Layer variants over the years, each offering progressively higher data rates:

Standard	Technique	Band	Typical Data Rate(s)
802.11 FHSS	Frequency-Hopping Spread Spectrum, FSK modulation	2.4 GHz ISM (79 subbands of 1 MHz)	1 or 2 Mbps
802.11 DSSS	Direct Sequence Spread Spectrum, PSK modulation	2.4 GHz ISM (1 Mbaud/s)	1 or 2 Mbps (BPSK/QPSK)
802.11 Infrared	Pulse Position Modulation (PPM)	Infrared light, 800-950 nm	1 or 2 Mbps
802.11a	OFDM (52 subbands: 48 data + 4 control), PSK/QAM	5 GHz ISM	Up to 54 Mbps
802.11b	High-Rate DSSS (HR-DSSS), CCK encoding	2.4 GHz ISM	1, 2, 5.5, 11 Mbps
802.11g	OFDM + forward error correction	2.4 GHz ISM	22 or 54 Mbps
802.11n	MIMO (multiple antennas, Tx and Rx)	2.4 / 5 GHz	Up to 600 Mbps (implementation-dependent)

Of particular note: 802.11a uses OFDM (Orthogonal Frequency-Division Multiplexing) specifically because it is highly effective at reducing the impact of interference, by spreading the signal energy across many narrow, mutually-orthogonal subcarriers. 802.11b is backward-compatible with the earlier DSSS specification. 802.11g offers backward compatibility with 802.11b, while raising the achievable data rate considerably. 802.11n was developed specifically to increase both speed and throughput while reducing unnecessary overhead, principally through the use of MIMO (Multiple-Input Multiple-Output) technology, which employs multiple antennas at both the transmitter and receiver to improve overall performance and to reduce the practical impact of noise.

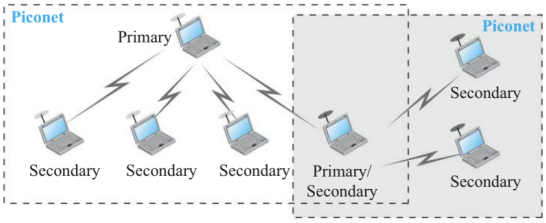
23. Bluetooth

Bluetooth is a short-range wireless technology used to connect devices such as mobile phones, computers, keyboards, mice, cameras, and printers to one another. It forms what is called an ad hoc Wireless Personal Area Network (WPAN), in which devices automatically discover one another and connect, generally without requiring any pre-existing network infrastructure. A Bluetooth network of this kind is specifically referred to as a piconet. Bluetooth is mainly used for short-range communication between nearby devices — common applications include wireless peripherals, healthcare sensors, home-security devices, and general device-to-device synchronization.

23.1 Bluetooth Architecture

Bluetooth supports two types of network arrangement: the Piconet, and the Scatternet.

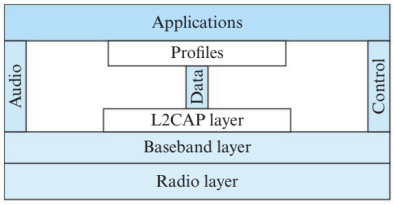
Figure 6.20 *Scatternet*



Bluetooth Layers

Bluetooth uses several layers that do not exactly match those of the Internet model we have defined in this book. Figure 6.21 shows these layers.

Figure 6.21 *Bluetooth layers*



Bluetooth architecture: a piconet has one primary and up to seven active secondary devices; multiple piconets combine to form a scatternet.

Piconet

A piconet can contain up to 8 active devices in total — exactly one primary device, and up to 7 secondary devices. The secondary devices all synchronise themselves to the primary device's clock and hopping sequence. Additional devices beyond the 7 active secondaries can be placed into a parked state, remaining associated with the piconet without actively participating, and can be switched to an active state later on, whenever they are actually needed.

Scatternet

Two or more separate piconets can be combined together to form what is called a scatternet. A single device is able to participate in two different piconets simultaneously, and in doing so, it may act as a secondary device within one piconet while simultaneously acting as the primary device of another piconet — effectively serving as a bridge that links the two piconets together.

23.2 Bluetooth Devices

A Bluetooth device incorporates a short-range radio transmitter and operates within the 2.4 GHz ISM band. The basic (classic) Bluetooth technology described here supports a data rate of roughly 1 Mbps. Because Bluetooth and IEEE 802.11b (Wi-Fi) both happen to operate within the same 2.4 GHz band, some mutual interference between the two technologies is possible when they are both in active use in close proximity.

23.3 L2CAP — Logical Link Control and Adaptation Protocol

The Logical Link Control and Adaptation Protocol (L2CAP) is a Bluetooth protocol that sits above the lower-level Bluetooth controller (the radio and baseband hardware), and provides communication services to the various higher-layer protocols and applications that run on top of it.

Figure 6.22 *L2CAP data packet format*



L2CAP sits above the Baseband layer in the Bluetooth protocol stack, providing multiplexing, segmentation/reassembly, and QoS support.

L2CAP is mainly used for data exchange over ACL (Asynchronous Connection-Less) links, and it does not carry synchronous SCO/eSCO traffic (which is instead handled directly by lower layers, since it has strict, continuous timing requirements). L2CAP provides several important services: protocol multiplexing (allowing several different upper-layer protocols to share the one underlying Bluetooth link at the same time), segmentation and reassembly (breaking large data units into smaller pieces suitable for the lower layer, and reassembling them at the far end), flow/error control, Quality of Service (QoS) support, and logical channel management. The Channel Identifier (CID) is what identifies the particular logical channel through which any given piece of data is actually being exchanged.

L2CAP allows data coming from several different upper-layer protocols to share the very same underlying Bluetooth link, while still ensuring that each piece of received data is correctly delivered up to its own appropriate protocol. Whenever a data unit is too large to be carried directly by the lower layer in one piece, L2CAP divides it into smaller segments for transmission at the sender's end, and reassembles those segments back into the original data unit at the receiver's end. It can also convey Quality of Service (QoS) information, tailored to the particular requirements of whatever communication is taking place.

23.4 Baseband Layer

The Baseband layer is the lower-level Bluetooth layer directly responsible for actually transferring packets over the wireless link. In classic Bluetooth, communication is organised around fixed time slots of 625 microseconds each, with transmission activity carefully coordinated between the primary device and its secondary device(s).

24. WiMAX

WiMAX (Worldwide Interoperability for Microwave Access) is a broadband wireless technology based on the IEEE 802.16 family of standards, specifically designed to provide wide-area wireless broadband access and last-mile connectivity — that is, the final connection stretch linking an individual subscriber's premises to the wider network backbone. WiMAX supports both fixed and mobile users, and it is capable of providing communication over considerably larger distances than a typical Wi-Fi network can reach.

WiMAX relies on a Base Station (BS) to communicate with multiple Subscriber Stations (SS) within its coverage area. The BS manages the overall communication and allocates transmission resources to individual subscriber stations using a scheduling mechanism, which means that individual stations do not need to continuously contend with one another for access to the channel (unlike the contention-based methods, such as CSMA, discussed earlier in this module for LANs).

The Base Station itself contains the radio and antenna systems needed for both transmitting and receiving signals. WiMAX is able to make use of a range of advanced techniques — including adaptive antennas,

OFDM/OFDMA, and MIMO — in order to improve coverage, capacity, and resistance to interference. Subscriber Stations, for their part, provide connectivity directly to end users and may be installed either indoors or outdoors, while genuinely portable subscriber devices can additionally support fully mobile WiMAX services.

At the MAC layer, WiMAX relies on scheduling, and it supports differentiated Quality of Service (QoS) for different categories of traffic (for instance, giving voice or video traffic more predictable, timely access than ordinary best-effort data traffic). At the Physical Layer, IEEE 802.16 technologies are generally based on OFDM techniques, while mobile WiMAX specifically introduced Scalable OFDMA (SOFDMA), together with explicit support for user mobility. Overall, WiMAX finds application in broadband Internet access, in extending connectivity to rural or otherwise under-served areas, and more generally in wide-area wireless communication scenarios where a technology with greater range than Wi-Fi is required.

25. Mobile IP

Mobile IP is an extension of the Internet Protocol (IP) that allows a mobile computer to remain connected to the Internet, and to keep receiving packets, even while it moves from one network to another. With the enormous growth of laptops, tablets and smartphones, users no longer stay attached to one fixed network; they carry their devices from a home LAN to a college campus, an airport or another city. Ordinary IP was designed for stationary hosts and cannot support this kind of movement without help, and Mobile IP is the mechanism that provides that help.

The central idea is simple: the mobile host keeps one permanent address that the rest of the Internet always uses to reach it, and a pair of cooperating routers (the *agents*) forward its packets to wherever the host currently happens to be. The remote party never needs to know that the host has moved.

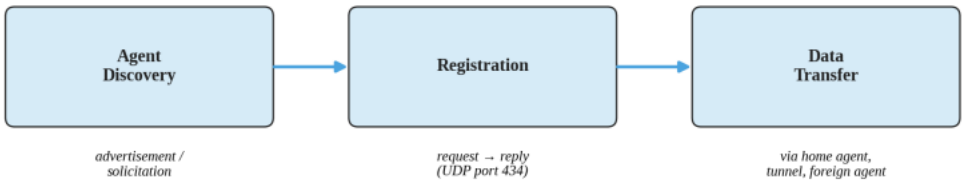


Figure M1 (added) The three phases of Mobile IP: agent discovery, registration and data transfer.

25.1 Addressing

The main problem that Mobile IP has to solve is addressing. To understand why, we first look at how addressing works for a stationary host and then at what goes wrong when the host moves.

Stationary Host

In normal IP, a host is connected to a fixed network and its IP address has two parts: the **network part (prefix)**, which identifies the network, and the **host part (suffix)**, which identifies the host within that network. For example, the address 10.3.4.24/8 belongs to the network 10.0.0.0/8. Routers all over the Internet forward packets by looking only at the prefix, so the address is tied firmly to one particular network.

Because of this, if the host physically moves to another network but keeps its old address, packets addressed to it are still routed to the *old* network and never reach it. Simply changing the address is also not a good answer: the host would lose all existing connections, and other computers that know the old address (through DNS entries, for example) would be unable to find it. Advertising a separate route for every moving host is equally impractical, since it would make the routing tables of Internet routers grow enormously. Mobile IP avoids both problems by giving the mobile host two addresses.

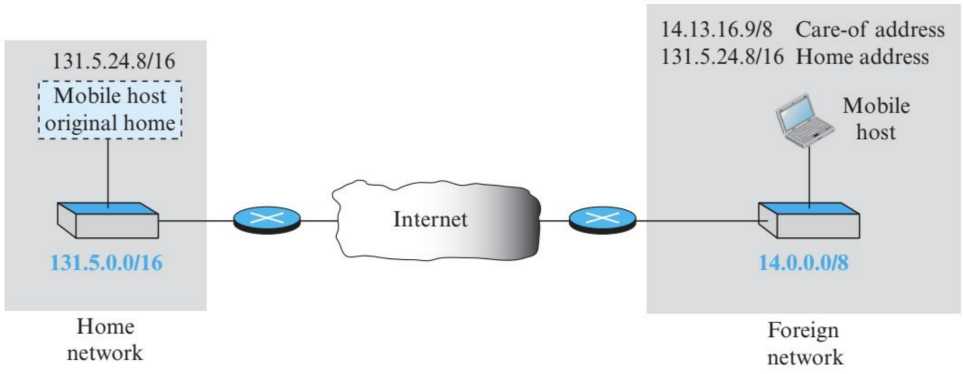
Mobile Host

When a host moves from one network to another, it uses two addresses:

- **Home Address** – the permanent address of the mobile host. It belongs to the host's *home network* (its original network) and never changes, no matter where the host goes. All other Internet hosts use this address to talk to the mobile host.

- **Care-of Address** – a temporary address obtained by the mobile host when it visits a *foreign network*. It identifies the mobile host's current location and changes every time the host moves to another network. Only the mobile host and its agents make use of it; the remote hosts never see it.

Figure 6.52 *Home address and care-of address*



Home address and care-of address: the mobile host keeps its home address (131.5.24.8/16) and additionally receives a care-of address (14.13.16.9/8) in the foreign network.

In the figure, the mobile host's home network is 131.5.0.0/16 and its home address is 131.5.24.8/16. When it moves to the foreign network 14.0.0.0/8, it obtains the care-of address 14.13.16.9/8 while still retaining 131.5.24.8/16 as its home address.

Feature	**Home Address**	**Care-of Address**
Nature	Permanent	Temporary
Belongs to	Home network	Foreign (visited) network
Changes when host moves?	No	Yes, at every new network
Used by	Remote hosts, to reach the mobile host	Home agent, for forwarding (tunneling) packets
Example (Fig. above)	131.5.24.8/16	14.13.16.9/8

25.2 Agents

To make the two-address scheme work, Mobile IP uses two cooperating agents, one in each of the networks involved. An agent is normally a router that has been given the extra task of supporting mobile hosts.

1. Home Agent (HA)

- Usually a router attached to the mobile host's *home network*.
- It acts as the representative of the mobile host whenever the host is away from home.
- It intercepts packets that are sent to the mobile host's home address, because it is the router that is directly connected to the home network.
- It forwards these packets to the foreign agent (or to the collocated care-of address) by tunneling, and it keeps a record of the mobile host's current care-of address, learned through registration.

2. Foreign Agent (FA)

- Usually a router attached to the *foreign network* that the mobile host is visiting.

- It receives the tunneled packets from the home agent and delivers them to the mobile host.
- It helps the mobile host to obtain and register its care-of address; the address of the foreign agent itself is commonly used as the care-of address, which is shared by all visiting hosts.
- It also acts as an ordinary router for packets that the visiting mobile host sends out.

Collocated Care-of Address

A mobile host can also act as its own foreign agent. In this case the host obtains a temporary address in the foreign network by itself (for example through DHCP), and that address is called a **collocated care-of address**. The advantage is that the mobile host does not depend on a foreign agent being present in the visited network; the disadvantage is that the host itself must run the extra software needed to receive tunneled packets.

	Home Agent	**Foreign Agent**
Located in	Home network	Foreign network
Main role	Represents the absent mobile host; intercepts and tunnels its packets	Delivers tunneled packets to the visiting mobile host
Knows	Home address to care-of address binding	Which mobile hosts are currently visiting
Typical device	Router	Router

25.3 Three Phases of Mobile IP

A communication between a remote host and a mobile host that has moved to a foreign network involves three distinct phases, which occur in this order:

`Agent Discovery` → `Registration` → `Data Transfer`

25.3.1 Agent Discovery

The mobile host must first find out which agents are available. It discovers three things: its **home agent**, which it learns of before leaving the home network; a **foreign agent**, once it has moved into a foreign network; and its **care-of address** in the foreign network. Discovery is done with two messages, which are built by extending the standard ICMP router-discovery messages:

- **Agent Advertisement** – an agent (home or foreign) announces its presence on the network by broadcasting or multicasting this message at regular intervals. It carries the address of the agent, the care-of address offered, the type of agent, and the lifetime for which the information is valid.
- **Agent Solicitation** – if a mobile host does not receive an advertisement, or does not want to wait for the next periodic one, it sends a solicitation message to ask any agent on the link to identify itself. The agent then replies with an advertisement.

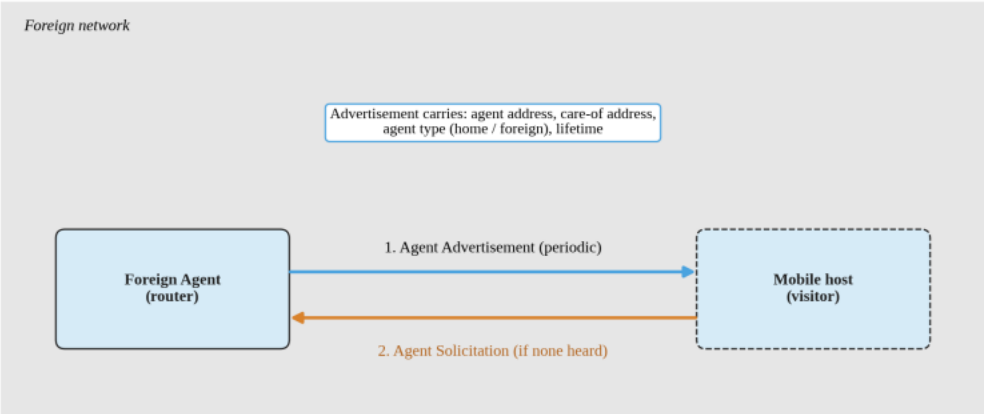


Figure M2 (added) *Agent discovery: a foreign agent periodically advertises itself; a mobile host that hears nothing sends an agent solicitation.*

By comparing the advertisements it hears, the mobile host also learns whether it is currently in its home network or in a foreign network. If it is at home, it simply behaves like an ordinary stationary host and no mobility service is needed.

25.3.2 Registration

After moving to a foreign network and obtaining a care-of address, the mobile host must **register** its new location, so that its home agent knows where to forward packets. The mobile host cannot always contact the home agent directly, hence registration normally goes through the foreign agent. The basic process is:

`Mobile Host` → `Foreign Agent` → `Home Agent`

The mobile host sends a **Registration Request** that contains information such as its home address, the address of its home agent, its care-of address, and the requested lifetime of the registration. The foreign agent relays this request to the home agent. The home agent updates its table, binding the home address to the new care-of address, and sends back a **Registration Reply**:

`Home Agent` → `Foreign Agent` → `Mobile Host`

The reply tells the mobile host whether the registration has been accepted or denied. Registration messages are carried inside UDP datagrams and use **UDP port 434**. Since a false registration could allow someone to steal another host's packets, the request and reply carry authentication information shared between the mobile host and its home agent. The registration is valid only for a limited lifetime, so a host that stays in a foreign network must renew it before it expires; when the host returns home it de-registers, and the home agent stops forwarding.

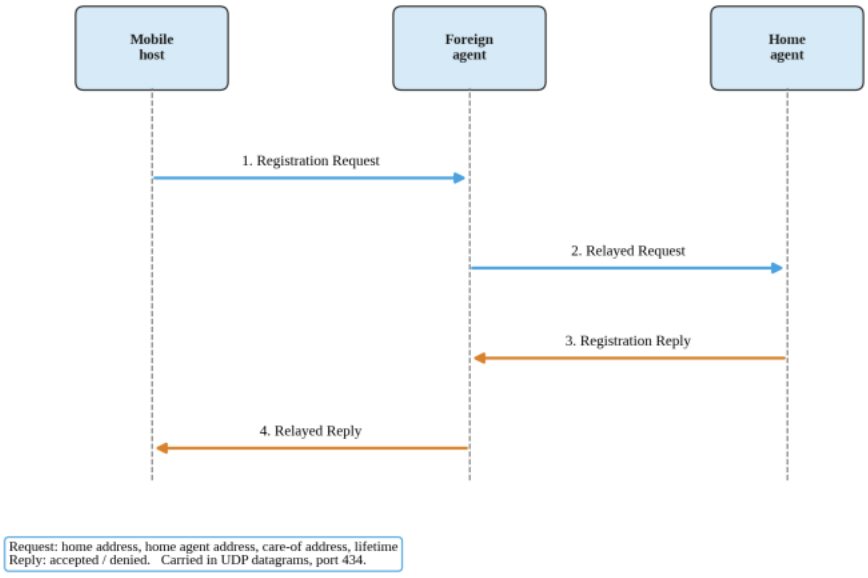
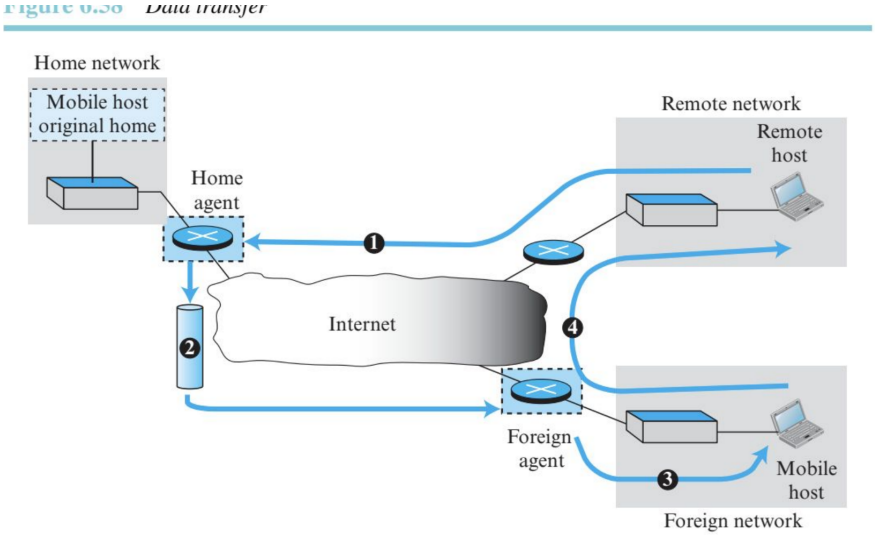


Figure M3 (added) *Registration: the request travels from the mobile host to the home agent via the foreign agent, and the reply returns along the same path.*

25.3.3 Data Transfer

Once registration is complete, the mobile host can communicate with a remote host. The exchange proceeds in four steps, shown by the numbered arrows in the figure below.



Data transfer: packets from the remote host reach the mobile host through the home agent, a tunnel and the foreign agent; replies go directly back.

- **Step 1: Remote Host → Home Agent.** The remote host sends its packet to the mobile host's *home address*, exactly as it would for a stationary host. The packet is routed to the home network, where the home agent intercepts it on behalf of the absent mobile host.

- **Step 2: Home Agent → Foreign Agent.** The home agent sends the packet to the foreign agent using **tunneling**. The original IP packet is encapsulated as the data of a new IP packet whose destination is the care-of address, so that ordinary routers on the Internet forward it without any special knowledge of mobility.
- **Step 3: Foreign Agent → Mobile Host.** The foreign agent removes the outer (tunnel) header to recover the original packet, uses its registration information to find the visiting mobile host, and delivers the packet to it.
- **Step 4: Mobile Host → Remote Host.** When the mobile host replies, it sends the packet in the normal way through the foreign network's router, using the *home address of the mobile host as the source* address and the *remote host's address as the destination*. The reply therefore does not need to pass through the home agent.

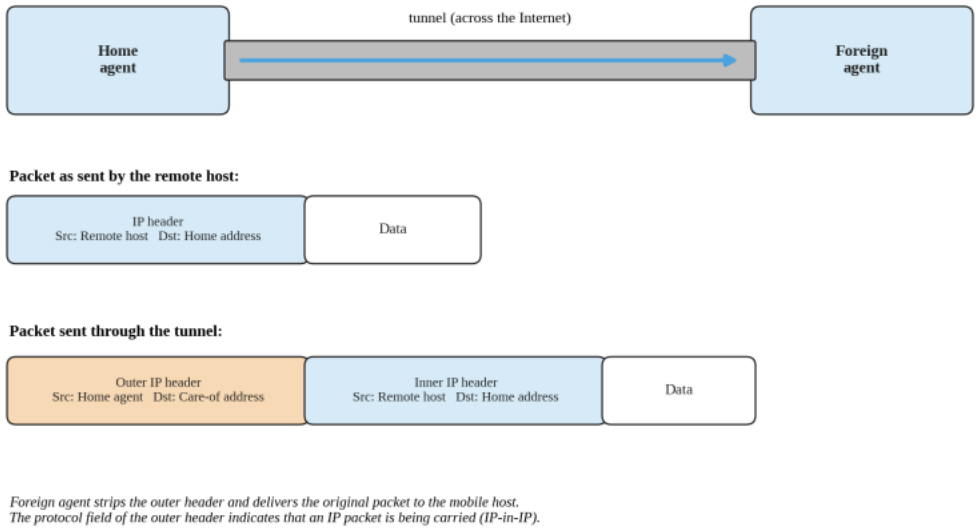


Figure M4 (added) Tunneling: the home agent adds an outer IP header (source = home agent, destination = care-of address) and the foreign agent removes it.

Transparency

The main advantage of Mobile IP is **transparency**. The remote host does not need to know that the mobile host has moved to another network; it continues to use the mobile host's home address for all communication. The complete flow is therefore:

`Remote Host → Home Agent → Foreign Agent → Mobile Host`

and for the return path: **Mobile Host → Remote Host**. Applications and transport-layer connections (such as an ongoing TCP session) continue to work even when the mobile host changes its network, because the addresses seen by the remote host never change.

> *Note: The path Remote Host → Home Agent → Foreign Agent → Mobile Host is often longer than the direct path, while the return path is direct. This asymmetry is called **triangle routing**, and it is the main inefficiency of basic Mobile IP: even when the remote host is physically close to the mobile host,*

its packets first travel to the home network and are then tunneled back. It can be reduced if the remote host is informed of the care-of address so that it can tunnel packets directly (route optimization).

Worked Example

Let the mobile host have home address 131.5.24.8/16, with its home agent in network 131.5.0.0/16, and let it move to the foreign network 14.0.0.0/8, where it obtains the care-of address 14.13.16.9/8. A remote host R sends a packet with source R and destination 131.5.24.8. The packet reaches the home network, and the home agent intercepts it and wraps it in an outer packet with destination 14.13.16.9. The foreign agent in network 14.0.0.0/8 receives this outer packet, removes the outer header, and hands the original packet (destination 131.5.24.8) to the mobile host. The mobile host's reply has source 131.5.24.8 and destination R.

25.4 Summary of Mobile IP

Phase	Purpose	Messages / Mechanism
1. Agent Discovery	Find home agent, foreign agent and care-of address	Agent Advertisement, Agent Solicitation (extended ICMP)
2. Registration	Tell the home agent the current care-of address	Registration Request / Reply over UDP port 434
3. Data Transfer	Deliver packets to and from the mobile host	Interception by home agent, tunneling, delivery by foreign agent

- **Advantages:** transparent to remote hosts and applications; the mobile host keeps a single permanent IP address; no changes are required in the routers of the rest of the Internet.
- **Limitations:** triangle routing makes the forward path longer; extra processing is needed for tunneling; the home agent is a critical point, since its failure prevents packets from reaching a mobile host that is away from home.